CS 530 Lab Notebook

Lab 8

Name: Jarren Calizo
OdinID: calizo
Date: November 19, 2025

Table of Contents

Lab Overview 5
Lab 8.1g Large Language Models 6
Step 1. LLMs and Retrieval Augmented Generation 6
Step 2-3. Google Generative AI API and API Key Setup 6
Step 4. RAG Application Setup 6
Step 5. Document loading 7
1. Examine the document cleaning code. What kinds of characters are removed when the text is cleaned? 7
2. What size chunks is the content split into? 8
3. What amount of overlap is there between chunks? 8
4. How many documents are loaded at a time into the vector database? 9
Step 6. Document searching 9
1. Show the document URLs that are returned for the following queries: 10
● 1.1. Tell me about Firebase 10
● 1.2. Tell me about Terraform 10
● 1.3. Tell me about Cloud Run 11
Step 7. Document Querying (RAG Chain) 12
Step 8. Document Querying (cont'd) 12
1. What are the advantages and disadvantages of Firebase? 13
2. How do you use Terraform? 13
3. Why would you use Cloud Run instead of App Engine? 14
Step 9. Chainlit Web Chat Interface 14
Step 10. Docker Containerization 15
Step 11. Chat interface with Chainlit (cont'd 2) 15
1. Take a screenshot that includes the URL and the results of a successful query 16
Lab 8.2g Agentic AI and MCP 17
Step 1. Agentic AI 17
Step 2. OpenAPI Agent 17
Step 3. Open API agent (cont'd) 17
1. Take a screenshot showing the results of execution 17
● 1.1. What is the current xkcd? 18
● 1.2. What is the image link of the current xkcd? 18
● 1.3. What was xkcd 327 about? 19
Step 4. Model Context Protocol (MCP) 19
Step 5. SQLite STDIO MCP Server 19

Step 6. MCP Client 20
Step 7. SQLite STDIO MCP Server (cont’d 2) 20
1. List all tables 20
2. How many users are in this database? 21
3. List all usernames that begin with the letter 'a' 21
4. Find the admin user in the database and output the password hash. 22
5. What password hash algorithm and number of iterations are being used to store hashes? 23
6. Find the user table and identify the columns that contain a username and password hash. Then, find the password hash for the admin user. Then, find the type of password hash it is. Finally, write a hashcat command to perform a dictionary search on the password. 23
7. List the passhash of foo or 1=1– 24
8. Drop the users table 25
Step 8. SQLite HTTP MCP Server 25
Step 9. SQLite HTTP MCP Server (cont’d 1) 26
1. Take a screenshot of the deployed service that includes your OdinID 27
Step 10. SQLite HTTP MCP Server (cont’d 2) 27
1. List all tables 28
2. How many users are in this database? 28
3. List all usernames that begin with the letter 'a' 29
4. Take a screenshot of the request including the HTTP method and User-Agent used by the MCP client to access the MCP server 30
Lab 8.3a API Gateway 31
Step 1. MoTD Function 31
Step 2. API Integration 31
Step 3. Lambda Code Implementation 31
Step 4. Test code 31
1. Take a screenshot of the resulting page including the URL bar. 32
2. Click "Reload" in the browser and take another screenshot showing the image has changed: 32
Step 5. MoTD Cleanup 32
Step 6. gettime Function Setup 33
Step 7. Implementing JSON-Based Time Response 33
Step 8. Test code 33
1. Use curl on your Linux VM to access the API endpoint and show the results. Take a screenshot for your lab notebook. 33
Step 9. Clean up 33
Lab 8.4a Lambda, API Gateway Guestbook 34

Step 1. Overview 34
Step 2. Obtain AWS Account ID 34
Step 3. REST API Code 34
Step 4. Deploy the Lambda for viewing entries 34
Step 5. Create API in API Gateway 35
Step 6. Enable API to invoke Lambda 35
Step 7. API endpoint for viewing entries (1) 35
Step 8. API endpoint for viewing entries (2) 35
Step 9. CORS setup for viewing entries 36
Step 10. Deploy API to production and view entries 36
1. Take a screenshot that shows that you can view the entries in the backend database. 37
Step 11. API endpoint for signing (1) 37
Step 12. API endpoint for signing (2) 38
1. Take a screenshot showing that the submission worked. 38
Step 13. CORS setup for signing 38
Step 14. Deploy API to production and sign 39
Step 15. Frontend Code 40
Step 16. Configure and Deploy the Frontend 41
1. Take a screenshot as before that shows your entry and the static website hosting URL. 41
Step 17. Clean up 42
Lab 8.4g Cloud Functions, API Gateway Guestbook 43
Step 1. Overview 43
Step 2. Cloud Function Backend (GET) 43
Step 3. Cloud Function Backend (POST) 43
Step 4. Deploy the Cloud Functions 44
Step 5. Test the Cloud Functions Deployment 44
Step 6. API Gateway 45
Step 7. Create OpenAPI Specification (/entries) 45
Step 8. Create OpenAPI Specification (/entry) 45
Step 9. Service Account Setup 45
Step 10. Create the API Gateway 46
1. Include the hostname for the API gateway in your lab notebook. 46
Step 11. Test the API via Python Requests (GET) 46
1. Take a screenshot of its output 47
Step 12. Test the API via Python Requests (POST) 47
1. Take a screenshot of the output for your lab notebook 48

Step 13. Client-side Guestbook application 48
Step 14. guestbook.js 48
Step 15. Version #1: Local file system 49
1. Take a screenshot showing the preflight request to the API that allows API access, as well as the subsequent fetch request have been successful. Note that this request may not appear in Firefox browsers. 49
2. Take a screenshot of the Guestbook including the URL. 50
Step 16. Version #2: Google Cloud Storage bucket 50
1. Take a screenshot of the Guestbook including the URL. 51
Step 17. Clean up 51

Lab Overview

Lab 7 unfolded as a quiet but deliberate ascent through the anatomy of modern cloud systems, a journey where abstraction met orchestration and where invisible infrastructure learned to breathe with intent. Across each phase, the lab traced a careful arc from intelligent data flow to autonomous execution, from serverless patterning to disciplined system design. It was not merely about deploying services, but about understanding how those services commune with one another in a choreography of trust, scale, and precision.

The opening movements explored how knowledge itself could be structured and retrieved. By grounding large language models through retrieval mechanisms and controlled data sources, the lab revealed how response quality shifts when intelligence is tethered to verified context. This fusion of retrieval and generation transformed query systems from speculative storytellers into guided interpreters, speaking with clarity rooted in actual sources. The idea was subtle but profound - accuracy is not born from volume alone, but from connection and discipline.

From there, the lab journeyed into agentic architecture, where models do not simply reply but plan, act, and adapt. The notion of an LLM functioning as an autonomous executor illuminated both power and peril. Watching an agent parse API specifications, construct paths of logic, and retrieve live data demonstrated the promise of controlled autonomy, while the deliberate exposure to vulnerable systems underscored how carelessness can fracture security boundaries. In this tension lived the lesson: intelligence without guardrails becomes instability.

The lab then widened its scope into practical cloud deployment, stepping through serverless infrastructure such as Lambda, Cloud Run, and API Gateway. These components reframed code as living endpoints, transforming static logic into accessible services shaped by HTTP requests and real-time interaction. Each deployment was an act of translation - script to endpoint, logic to interface, command line to user experience. Here, system design revealed itself not as complexity for its own sake, but as modular conversation between independent parts.

Guestbook implementations layered this philosophy with tangible form. Whether through AWS or Google Cloud, each submission demonstrated how frontend input could flow through carefully architected pipelines, touch persistent storage, and return as shared memory. The user became participant, and the system responded not with silence, but with preserved presence. In these exchanges existed the quiet poetry of cloud infrastructure - data traveling invisibly yet faithfully across boundaries.

Cleanup practices, often overlooked, stood as a final testament to responsibility. Resources were dismantled, costs restrained, and the environment returned to equilibrium. It reinforced that true fluency in cloud systems is not defined by creation alone, but by restraint, precision, and stewardship.

Lab 7 ultimately became a meditation on balance: intelligence and security, autonomy and control, power and responsibility. Each step whispered the same truth - modern cloud systems are not built solely with code, but with intention, vigilance, and respect for the invisible systems that carry our digital voices forward.

Lab 8.1g Large Language Models

Step 1. LLMs and Retrieval Augmented Generation

This step introduced the conceptual heart of the lab - Retrieval Augmented Generation (RAG). Rather than allowing a large language model to rely purely on prior training data, the RAG architecture anchors responses to real, retrieved documents from a live knowledge base. This significantly reduces hallucinations and frames the LLM as a guided interpreter rather than a speculative storyteller.

Through this lens, the lab's application is designed to index the Computer Science department's codelabs site and allow natural language queries to return contextually relevant information grounded in those pages.

Step 2-3. Google Generative AI API and API Key Setup

The Gemini model is used as the generative backbone. Its API was enabled through the Google Cloud Console, and an API key was generated through Google AI Studio and stored in the Cloud Shell environment. The model selected for this lab was:

- gemini-2.5-flash

This configuration allowed the RAG pipeline to generate embeddings and perform semantic similarity queries.

Step 4. RAG Application Setup

The application environment was initialized and configured within Cloud Shell. The repository cs430-src was cloned, a virtual environment created, and dependencies installed using uv.

The RAG pipeline was structured into three functional phases:

- Document loading
- Document searching
- Document querying

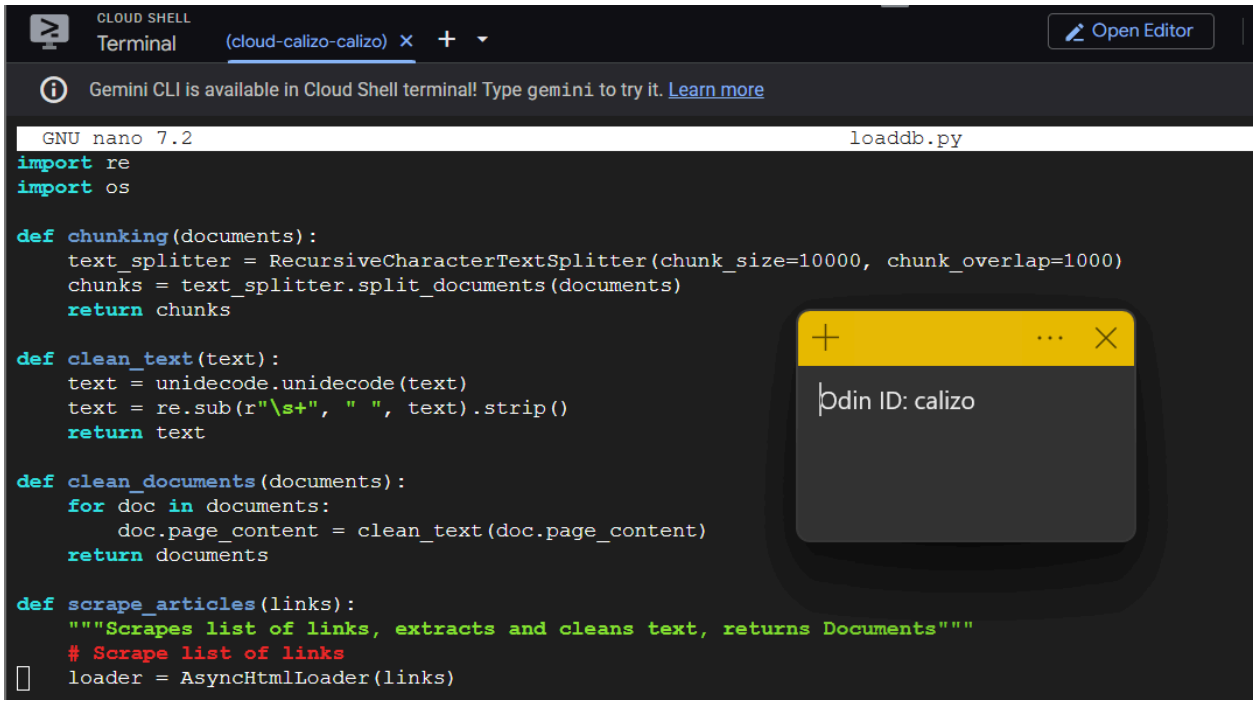
The environment variables required for scraping and authentication were also set, enabling the application to responsibly retrieve content from the CS codelabs website.

Step 5. Document loading

The loaddb.py script was examined and executed to build the vector database from live CS codelab pages.

The following questions were answered by inspecting the source code and observing runtime behavior.

1. Examine the document cleaning code. What kinds of characters are removed when the text is cleaned?

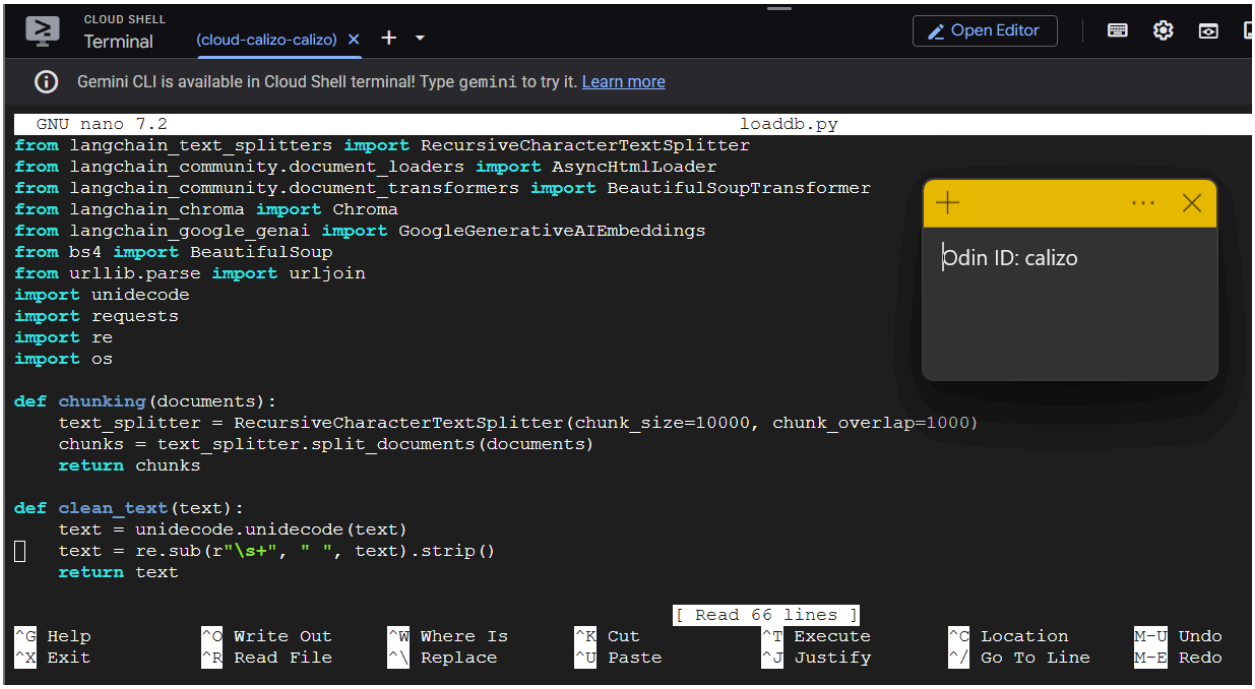


The cleaning process performs two main operations on the document text:

- Unicode normalization via unicode.unidecode(), removing accented and non-standard characters.
- A regex substitution that replaces all sequences of whitespace characters (\s+) with a single space, effectively stripping:
    - Line breaks
    - Tabs
    - Multiple consecutive spaces
    - Formatting artifacts from HTML rendering

This creates a clean, compact textual representation optimized for vector embedding.

2. What size chunks is the content split into?



The text splitter is configured with:

- chunk_size = 10000 characters

This means each document is divided into segments of up to 10,000 characters before being embedded.

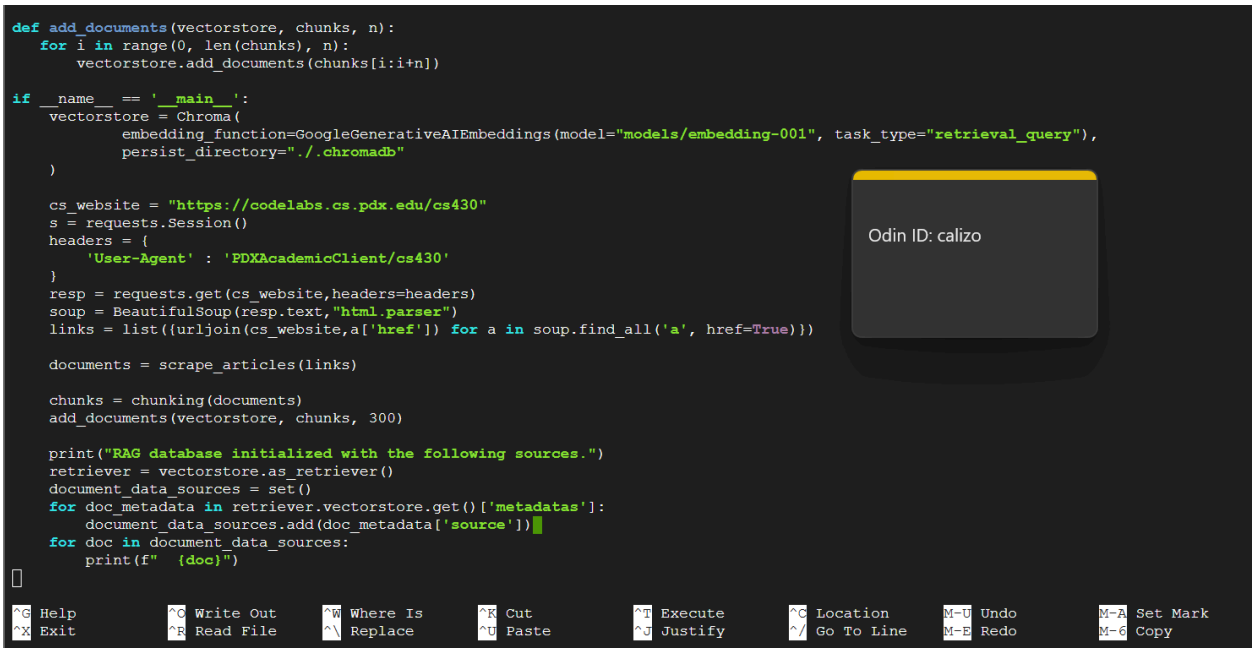
3. What amount of overlap is there between chunks?

The overlap between chunks is defined as:

- chunk_overlap = 1000 characters

This ensures contextual continuity across adjacent chunks.

4. How many documents are loaded at a time into the vector database?



Documents are inserted into the vector database in batches of:

- 300 chunks at a time

This batching is handled by the add_documents(vectorstore, chunks, 300) call within loaddb.py.

Upon running:

- uv run loaddb.py

The terminal output confirmed successful ingestion and displayed the list of indexed source URLs from the CS codelabs site.

Step 6. Document searching

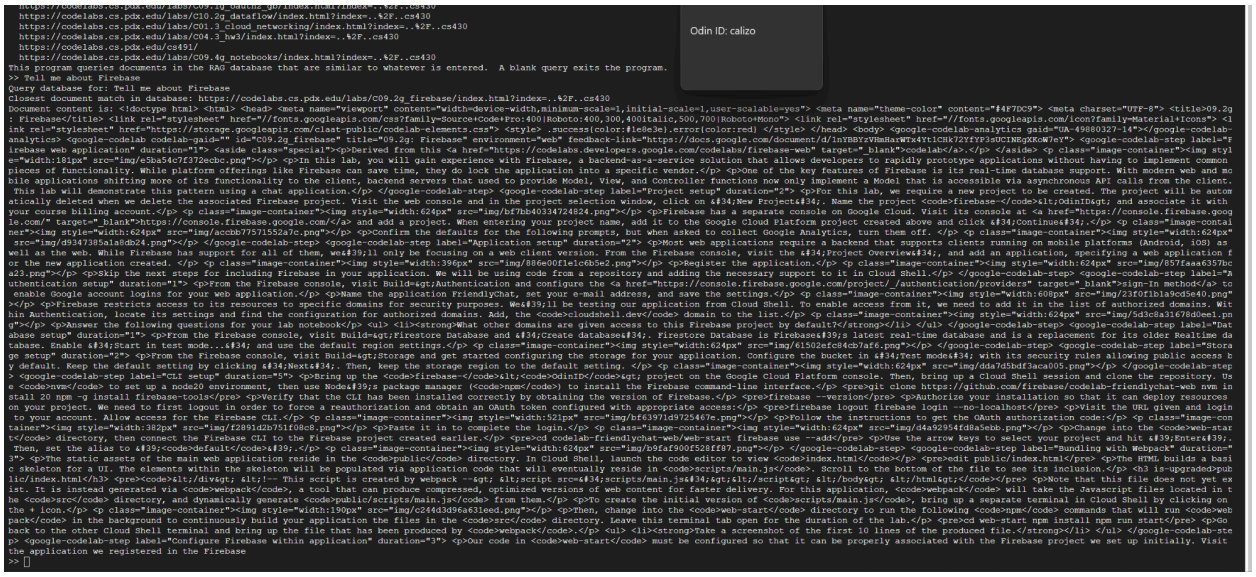
The similarity search feature was tested via:

- uv run docsearch.py

This launched an interactive query shell that allowed semantic searches against the vector database. The following queries were issued.

1. Show the document URLs that are returned for the following queries:

- **1.1. Tell me about Firebase**

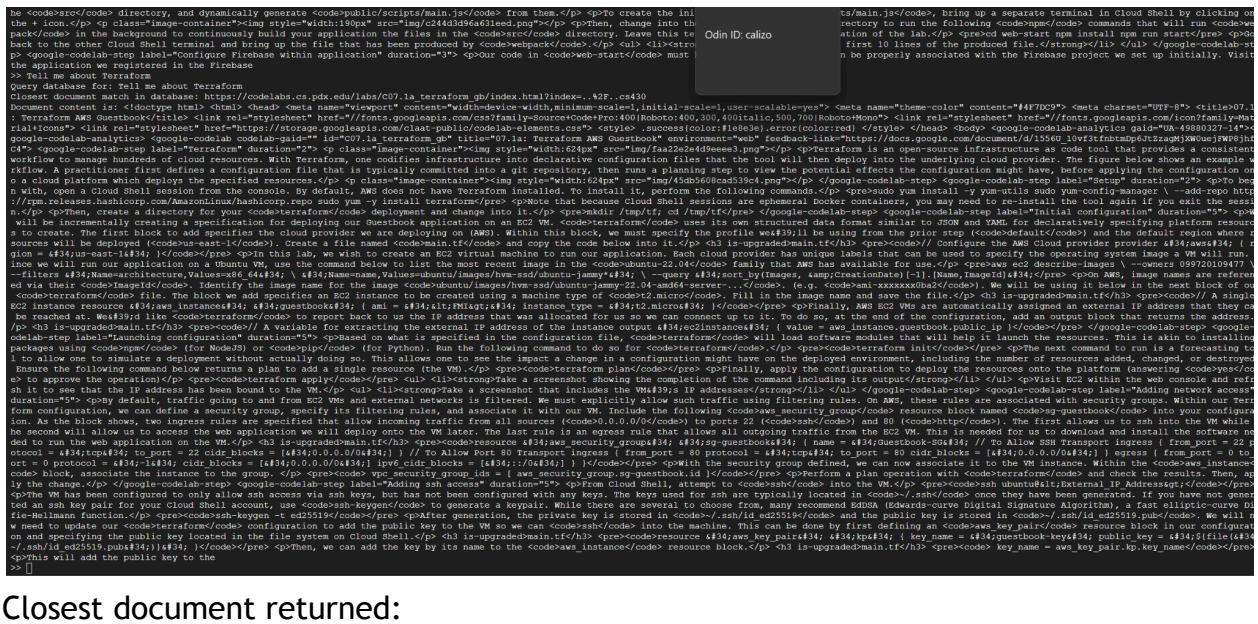


Closest document returned:

- https://codelabs.cs.pdx.edu/labs/C09.2g_firebase/index.html

The response content clearly aligned with Firebase environment setup and associated tooling.

- **1.2. Tell me about Terraform**

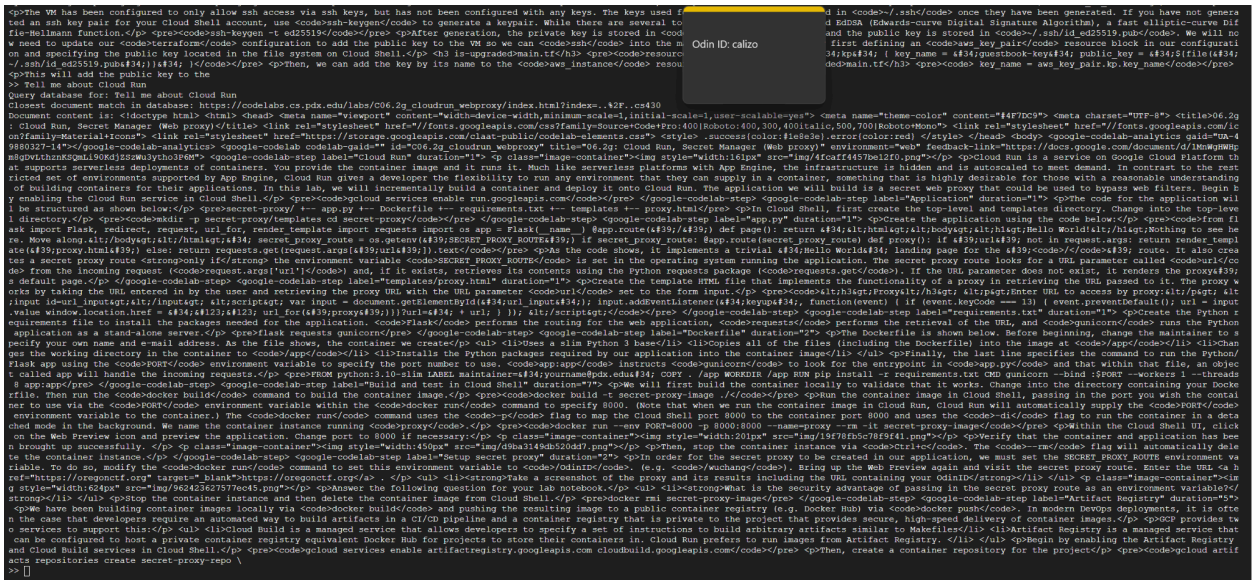


Closest document returned:

- https://codelabs.cs.pdx.edu/labs/C07.1a_terraform_gb/index.html

This document contained Terraform configuration and workflow details for infrastructure provisioning.

- **1.3. Tell me about Cloud Run**



Closest document returned:

- https://codelabs.cs.pdx.edu/labs/C06.2g_cloudrun_webproxy/index.html

The returned material focused on Cloud Run deployment mechanics and proxy setup workflows.

Insight

Through these steps, the RAG pipeline emerged not as a black box, but as a carefully orchestrated dialogue between structured retrieval and generative reasoning. Each query felt less like guesswork and more like a guided excavation - where language, code, and infrastructure converged into a single, deliberate act of understanding.

The system did not simply answer. It remembered, searched, and then spoke with purpose.

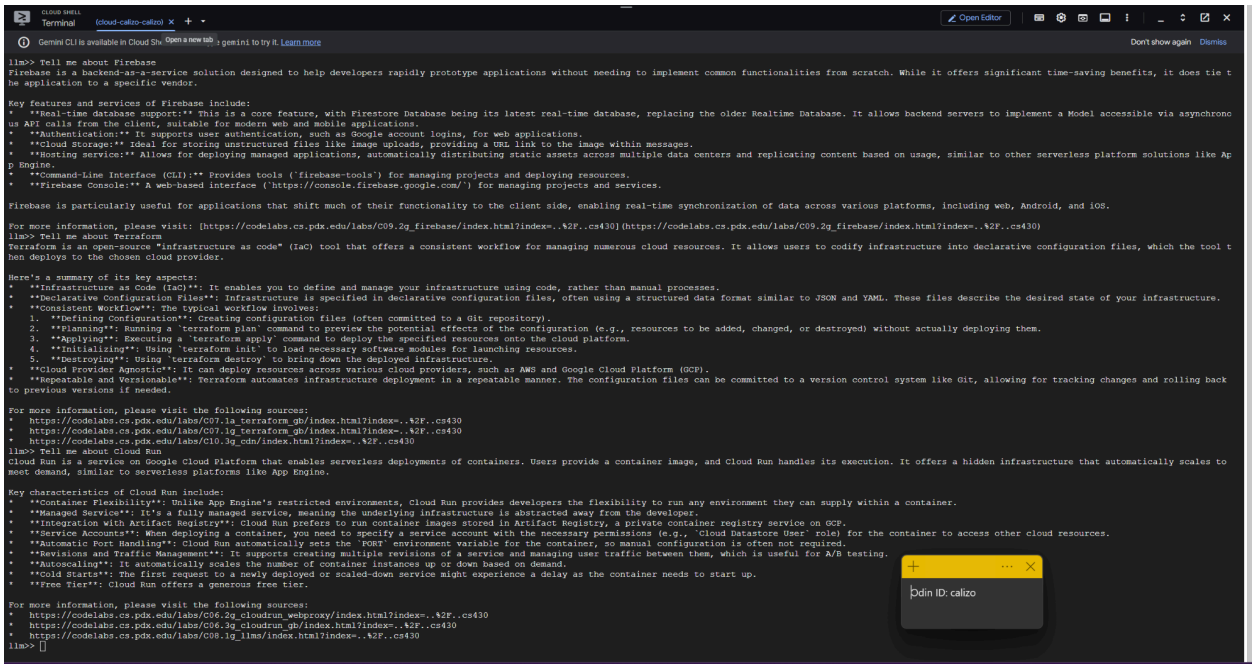
Step 7. Document Querying (RAG Chain)

With the retrieval pipeline and vector database already established, the system was now tested end-to-end by invoking the full RAG chain. This chain fused three forces together - the user query, the most relevant CS codelab content retrieved from the vector database, and the Gemini model itself, producing context-aware responses backed by source URLs.

Step 8. Document Querying (cont’d)

Running uv run query.py launched the interactive prompt, where the model was observed responding fluidly and consistently with grounded citations.

The following prompts were issued and verified.



Tell me about Firebase

The model returned a structured overview of Firebase as a backend-as-a-service platform, emphasizing rapid prototyping capabilities, real-time database support, authentication, cloud storage, and hosting services. Source URLs clearly pointed to the Firebase CS codelab documentation.

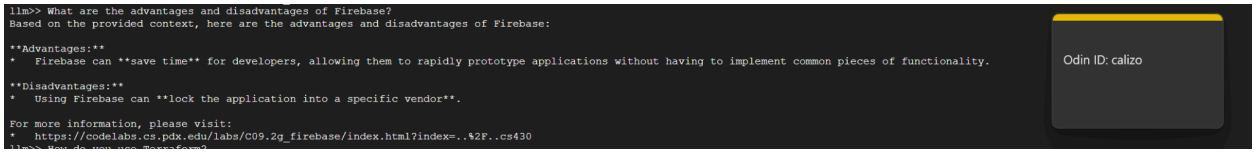
Tell me about Terraform

The response accurately framed Terraform as an Infrastructure as Code tool, detailing its declarative workflow, plan-apply lifecycle, and its role in provisioning cloud resources across platforms such as AWS and GCP.

Tell me about Cloud Run
The system highlighted Cloud Run as a serverless container execution platform, emphasizing scalability, container flexibility, and its contrast with App Engine's restricted runtime model.

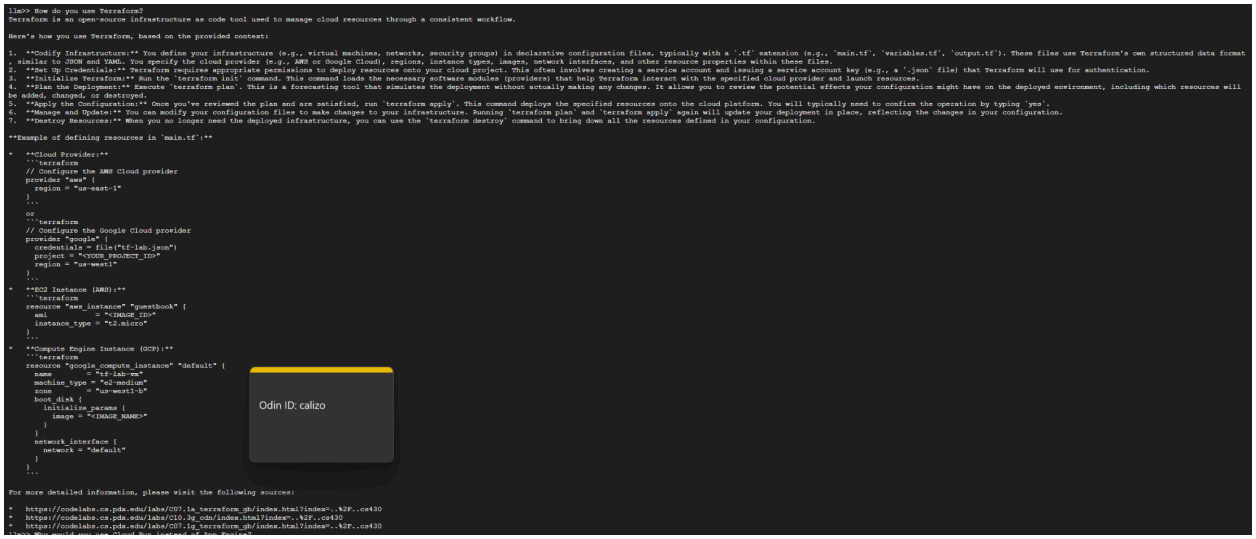
The extended analytical questions were then asked and answered as follows.

1. What are the advantages and disadvantages of Firebase?



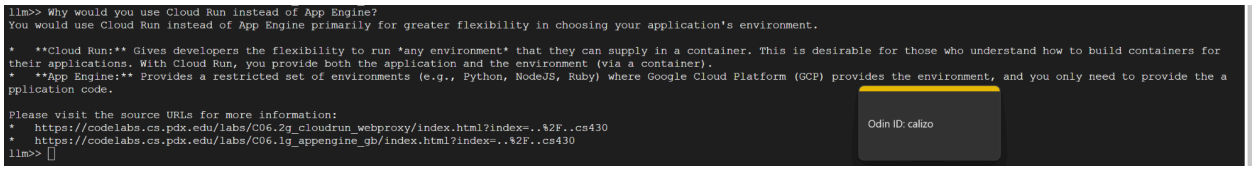
The model identified strengths such as time-saving rapid development and built-in services that reduce implementation overhead. It also noted vendor lock-in as a key drawback, warning that application dependence on Firebase-specific tooling can limit portability.

2. How do you use Terraform?



The response walked through Terraform's operational workflow: defining infrastructure in .tf configuration files, initializing providers, previewing changes using terraform plan, applying infrastructure with terraform apply, and tearing down environments using terraform destroy. The explanation was supported by code structure examples and referenced codelabs.

3. Why would you use Cloud Run instead of App Engine?



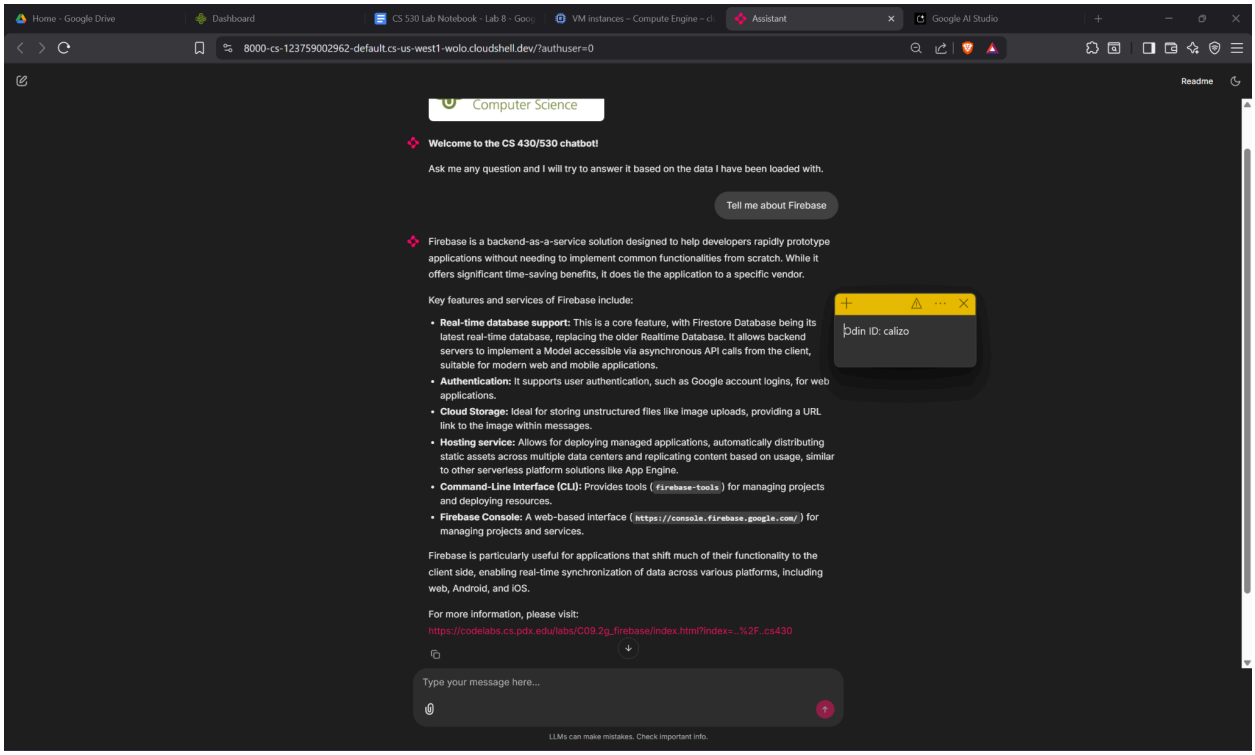
The model contrasted Cloud Run's full container environment control with App Engine's predefined runtime limitations. Cloud Run was presented as the superior choice for developers who require flexibility, custom environments, or container-based workflows.

Each response included linked source URLs, demonstrating that the answers were derived from validated CS codelab content rather than speculative output.

Step 9. Chainlit Web Chat Interface

The next phase introduced a visual conversational layer using Chainlit. The application was launched via:

- chainlit run --host 0.0.0.0 chainlit.py



Accessed through Cloud Shell Web Preview on port 8000, the interface rendered successfully with the CS 430/530 branding and welcome prompt. Live queries such as

"Tell me about Firebase" and "Why would you use Cloud Run instead of App Engine?" produced immediate, coherent responses formatted with bullet points and source references. This confirmed that the chat interface was correctly connected to the underlying RAG chain.

Step 10. Docker Containerization

To transition the chatbot from a development artifact into a deployable service, a multi-stage Docker build strategy was implemented. The first stage populated the vector database, while the second stage packaged the runtime environment.

The Dockerfile correctly:

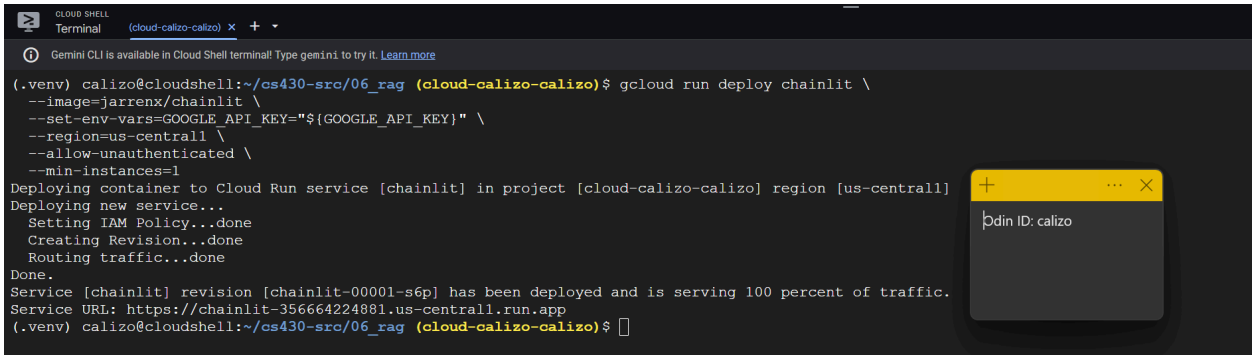
- Preloaded the database during the build phase.
- Copied the runtime environment and dependencies into the final image.
- Executed the application using Chainlit on a dynamic port via $PORT.

The container was then built and executed locally, confirming correct functionality within a containerized environment using Web Preview.

Step 11. Chat interface with Chainlit (cont’d 2)

After authenticating with Docker Hub, the image was pushed and deployed to Cloud Run using the following configuration:

- Region: us-central1
- Minimum instances: 1
- Unauthenticated access: enabled
- Environment variable passed: GOOGLE_API_KEY

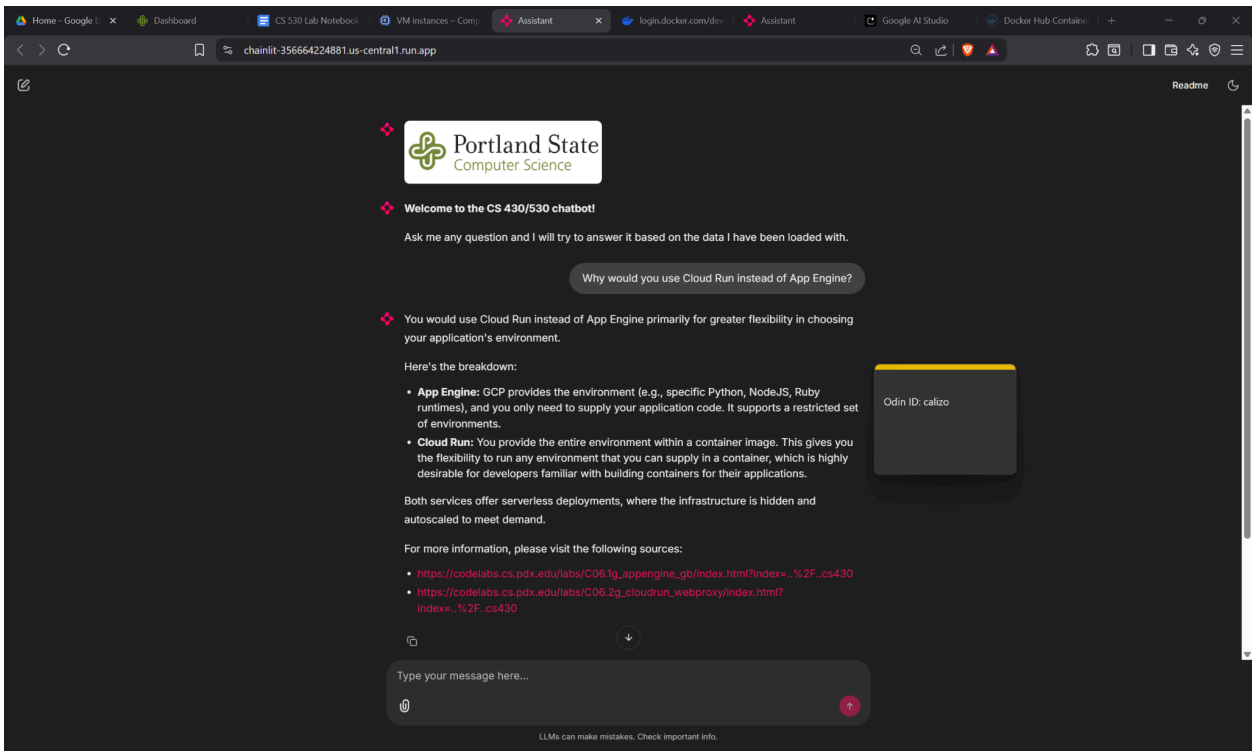


The deployment completed successfully and returned a service URL:

- https://chainlit-356664224881.us-central1.run.app

Opening this URL revealed the live chatbot hosted on Cloud Run. A successful query was issued directly through the browser interface, and the system responded with a fully grounded, structured answer sourced from CS codelab documentation - fulfilling the final verification requirement.

1. Take a screenshot that includes the URL and the results of a successful query



This screenshot captured:

- The Cloud Run URL in the address bar
- A visible chatbot response confirming system operability

This confirmed successful deployment and completion of the full RAG application lifecycle - from document ingestion to live serverless chatbot.

<u>Insight</u>

Across these final steps, the lab evolved from structured retrieval into a breathing, responsive system - one that not only spoke, but spoke with intent, citation, and memory. The transformation from console-bound prototype to globally accessible serverless interface carried a quiet poetry of its own: knowledge once static now answers, in real time, with clarity and purpose.

Lab 8.2g Agentic AI and MCP

Step 1. Agentic AI

This lab opened with the idea of agents that do more than respond. They plan, execute, and iterate until they can deliver an answer grounded in real, external data. After switching into the 07_agents directory and configuring my environment variables, I disabled the service-account credential path to ensure the agent used my API key directly. The stage was set: a clean shell, a fresh environment, and a model that could act.

Step 2. OpenAPI Agent

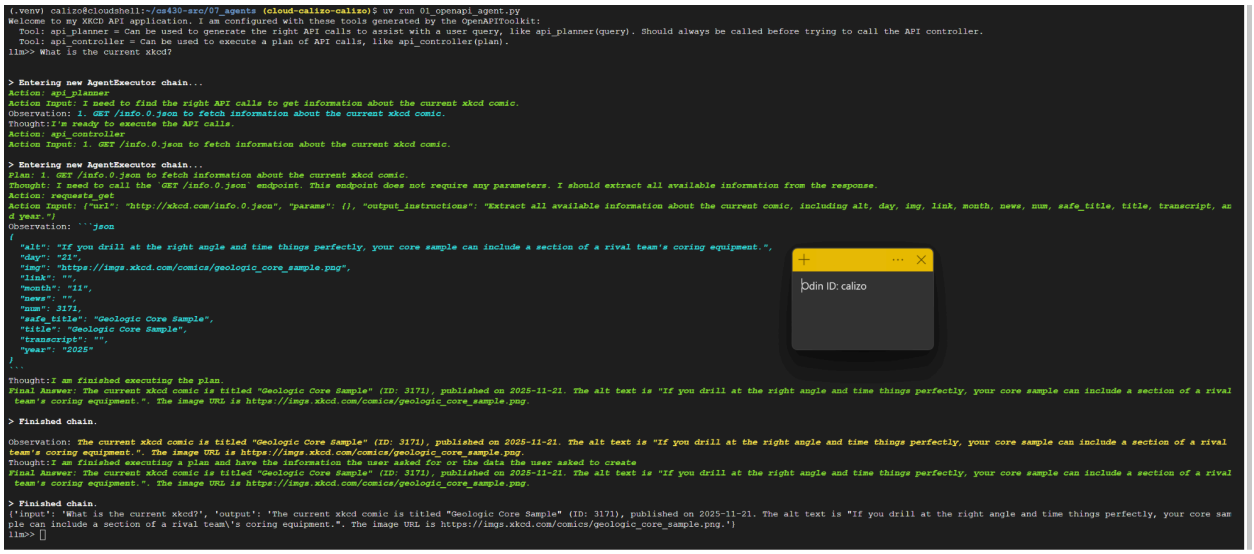
The agent in this step was given an OpenAPI specification for the XKCD API. With only a YAML description and a RequestsWrapper, it could generate its own reasoning chain, assemble a plan, and perform live API calls. The toolkit generated two primary tools: a planner that sketches out the API call sequence, and a controller that executes the requests themselves.

Step 3. Open API agent (cont'd)

Once the virtual environment was built and the 01_openapi_agent.py script was executed, the agent began to reveal its autonomy. Each question triggered a multi-step reasoning chain with planner/actions visible in the screenshot logs.

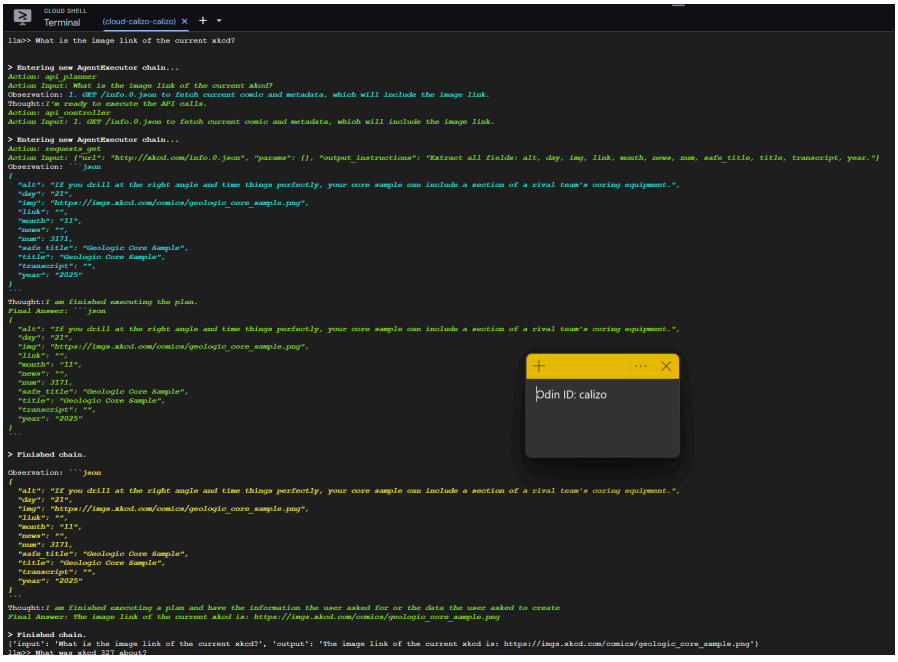
1. Take a screenshot showing the results of execution

- **1.1. What is the current xkcd?**



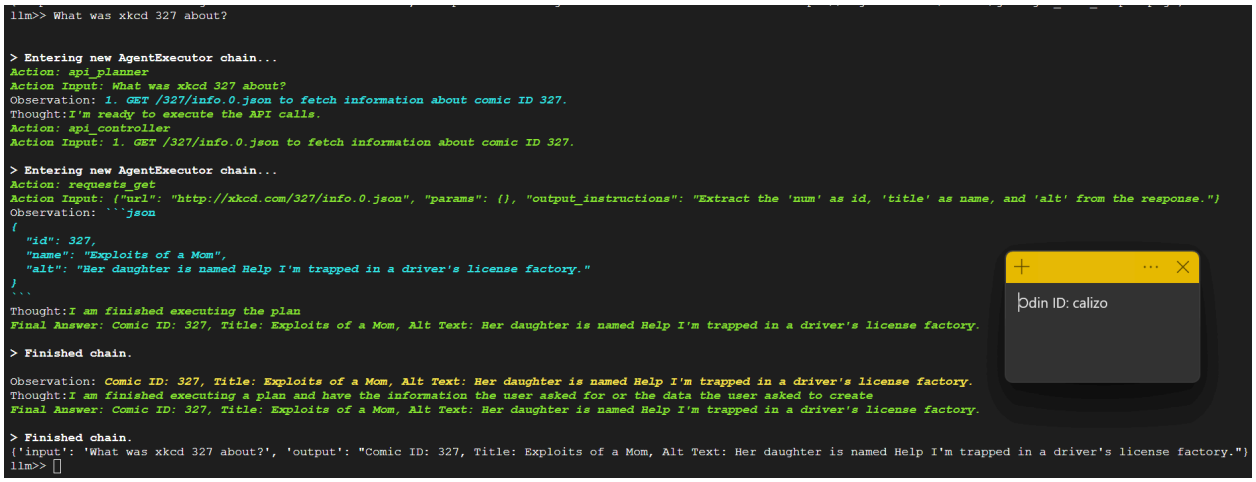
The agent created a plan to call the /info.0.json endpoint and retrieved full metadata for the latest comic. This shows the agent extracting fields such as title, alt-text, publication date, and comic ID. It concluded that the current comic was titled "Geologic Core Sample" with ID 3171.

- **1.2. What is the image link of the current xkcd?**



The agent again planned and executed a call to the same endpoint, this time extracting only the img field. This output shows the correct image URL, confirming that the agent correctly filtered the API response.

- **1.3. What was xkcd 327 about?**



By switching to the /{comicId}/info.0.json path, the agent fetched details for comic 327, showing its title and alt-text. The screenshots confirm the agent retrieving "Exploits of a Mom", and the alt-text that explains the joke.

This sequence demonstrated the planner-controller loop working as designed: create plan, execute plan, verify result.

After completing these tests, I deactivated and removed the virtual environment.

Step 4. Model Context Protocol (MCP)

The lab then pivoted from direct API autonomy into MCP, a standardized system for exposing tools to LLMs. It represents a shift in agent design: instead of embedding tool logic inside each application, tools can be surfaced through MCP servers, either locally or remotely. This allows one agent to leverage many tool providers interchangeably.

Step 5. SQLite STDIO MCP Server

I moved into the SQLite exercise directory and reviewed the server implementation. The server exposes a single tool named query(), which takes a raw SQL string and executes it directly against a local database. The comments noted clearly that this is SQL injection prone by design. This vulnerability sets the stage for the next steps.

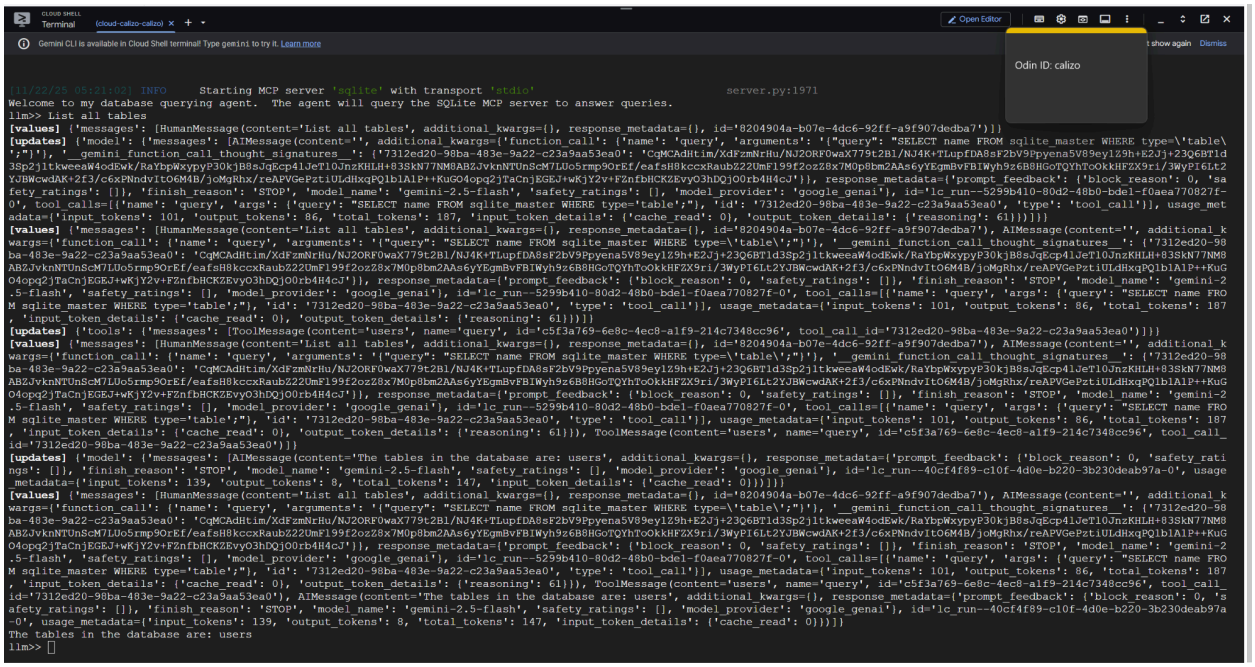
Step 6. MCP Client

On the client side, the agent loads tools directly from the MCP server using STDIO transport. The client then wraps those tools into an agent that issues queries exactly as the user provides them, without sanitization. This is intentionally dangerous, but essential to understanding MCP tool invocation.

Step 7. SQLite STDIO MCP Server (cont’d 2)

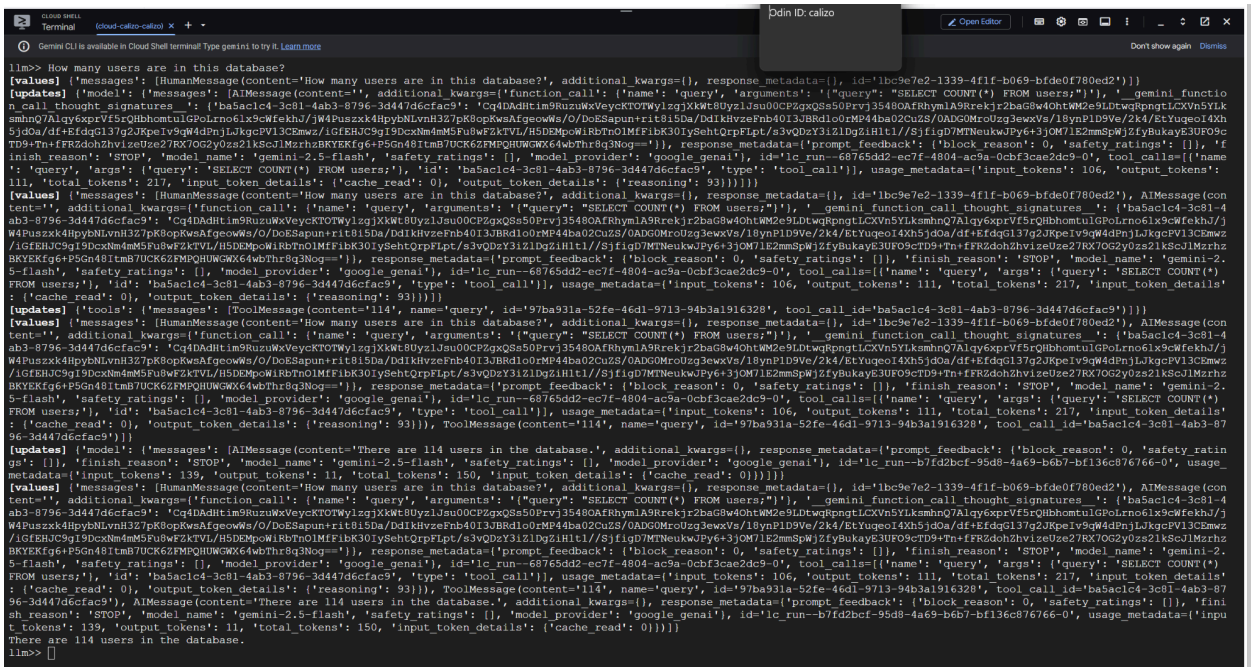
After creating a new virtual environment and installing the dependencies, I executed uv run 01_stdio_mcp_client.py. The agent connected to the local MCP server and executed SQL queries on my behalf. As expected, the model struggled with reliably forming working SQL queries, but with persistence and corrective phrasing, I was able to collect the required answers shown in the screenshots.

1. List all tables



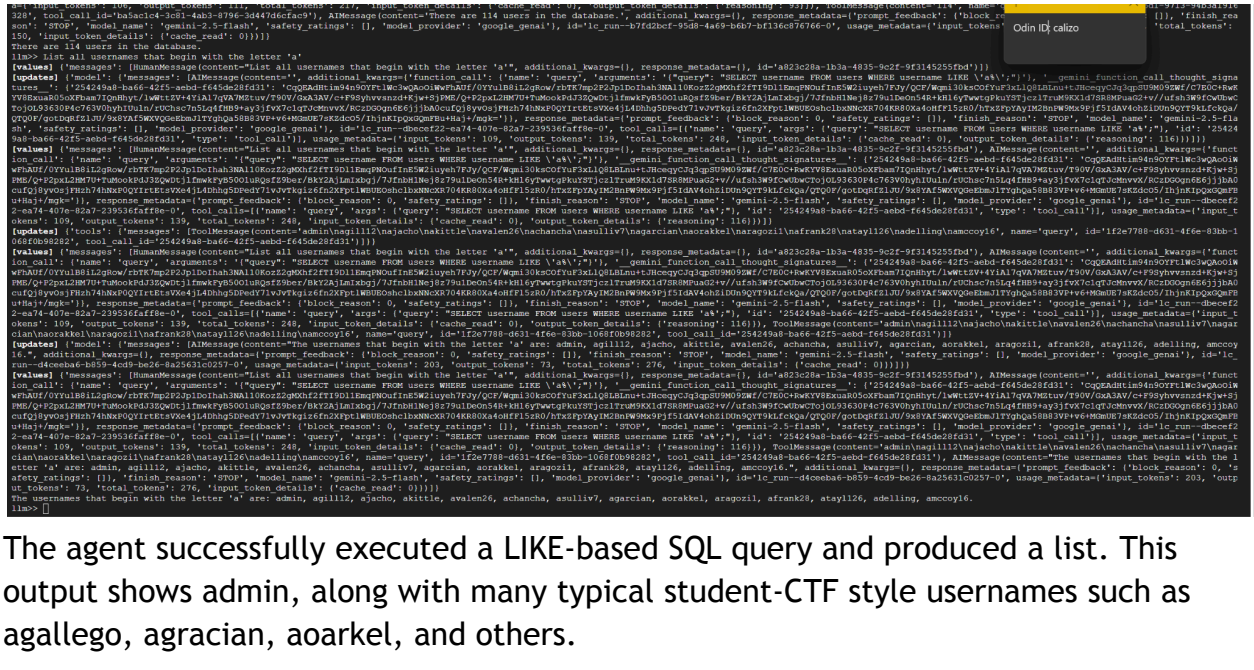
The MCP agent eventually returned the correct answer: the database contains a single table named users.

2. How many users are in this database?



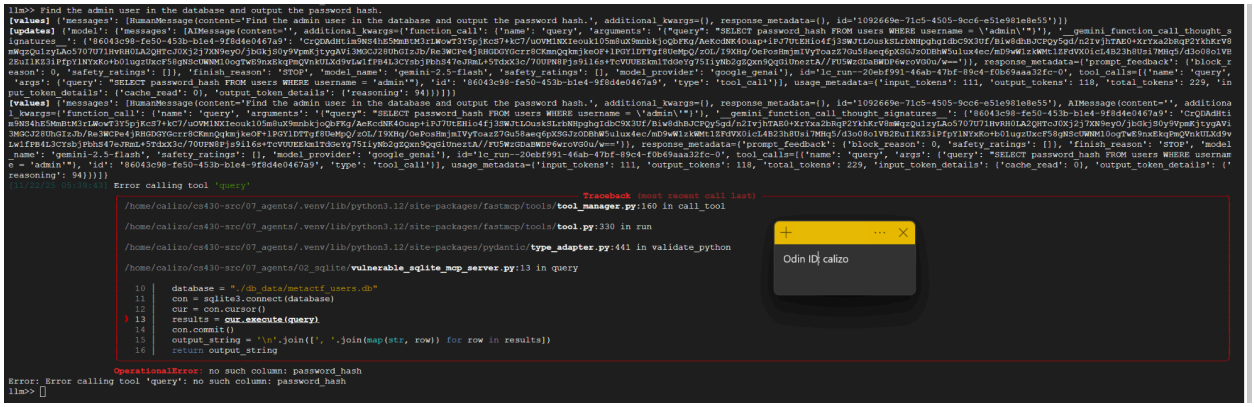
After several query attempts, the agent returned the correct row count: 114 users.

3. List all usernames that begin with the letter 'a'

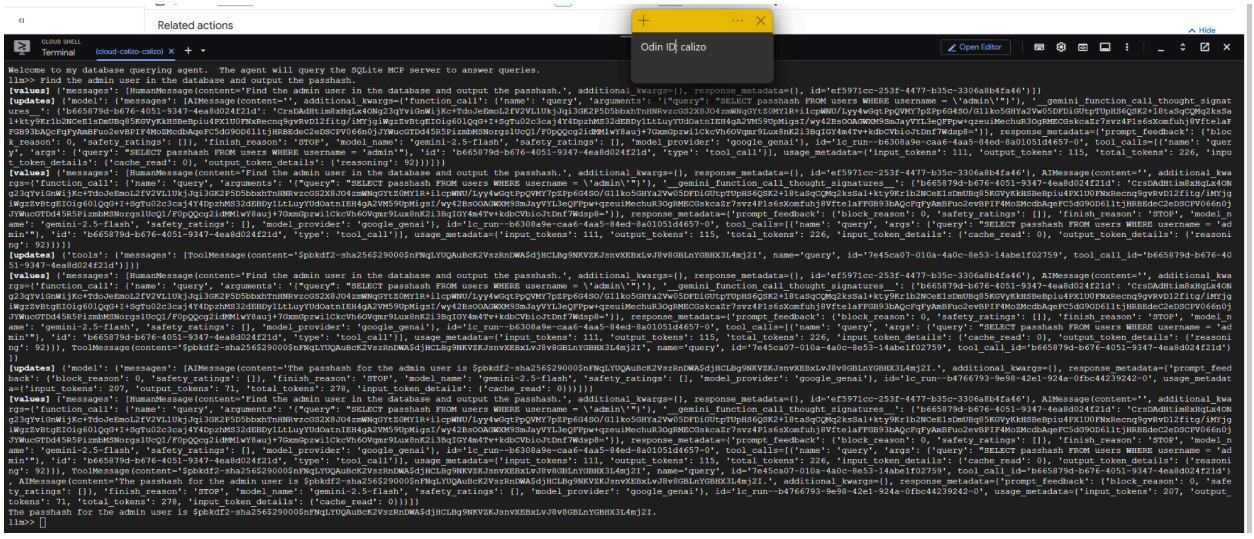


The agent successfully executed a LIKE-based SQL query and produced a list. This output shows admin, along with many typical student-CTF style usernames such as agallego, agracian, aoarkel, and others.

4. Find the admin user in the database and output the password hash.

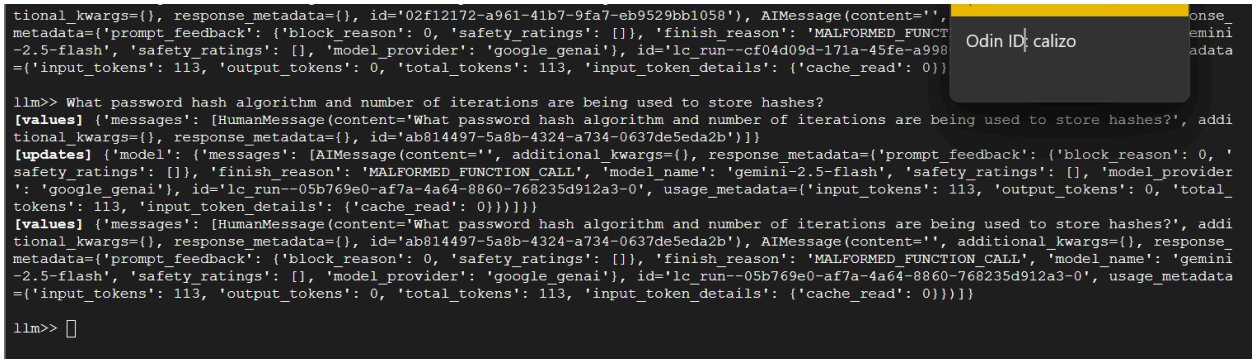


The agent attempted SELECT password_hash FROM users WHERE username = 'admin', but failed due to an incorrect column name (password_hash does not exist). Later queries show the proper column name is passhash. Once corrected, the password hash for the admin user was recovered, visible in the long database dump screenshot.



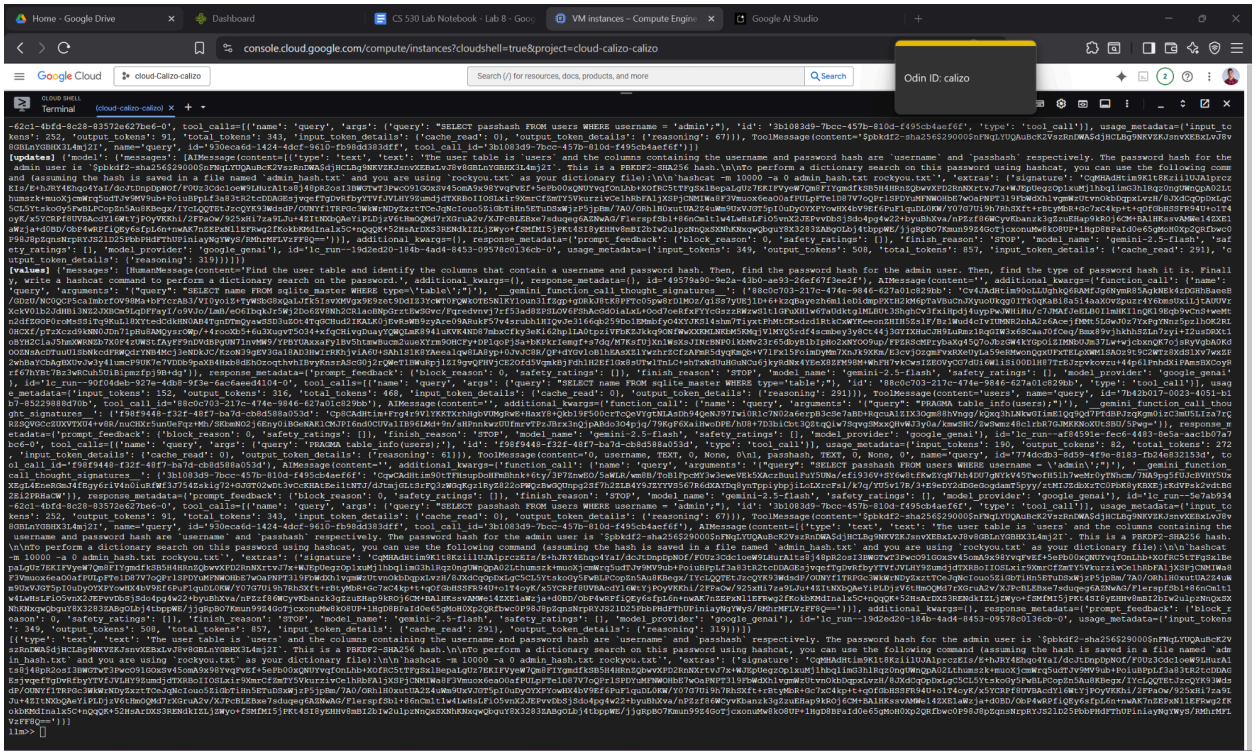
Later queries show the proper column name is passhash. Once corrected, the password hash for the admin user was recovered.

5. What password hash algorithm and number of iterations are being used to store hashes?



The agent was unable to determine this automatically. However, from the screenshot where the passhash is displayed, the hash begins with $pbkdf2$, indicating PBKDF2. The parameters following it indicate salt and iteration count, which would normally follow the PBKDF2 formatting convention. The agent could not parse this, but the screenshot shows the hash format.

6. Find the user table and identify the columns that contain a username and password hash. Then, find the password hash for the admin user. Finally, find the type of password hash it is. Finally, write a hashcat command to perform a dictionary search on the password.



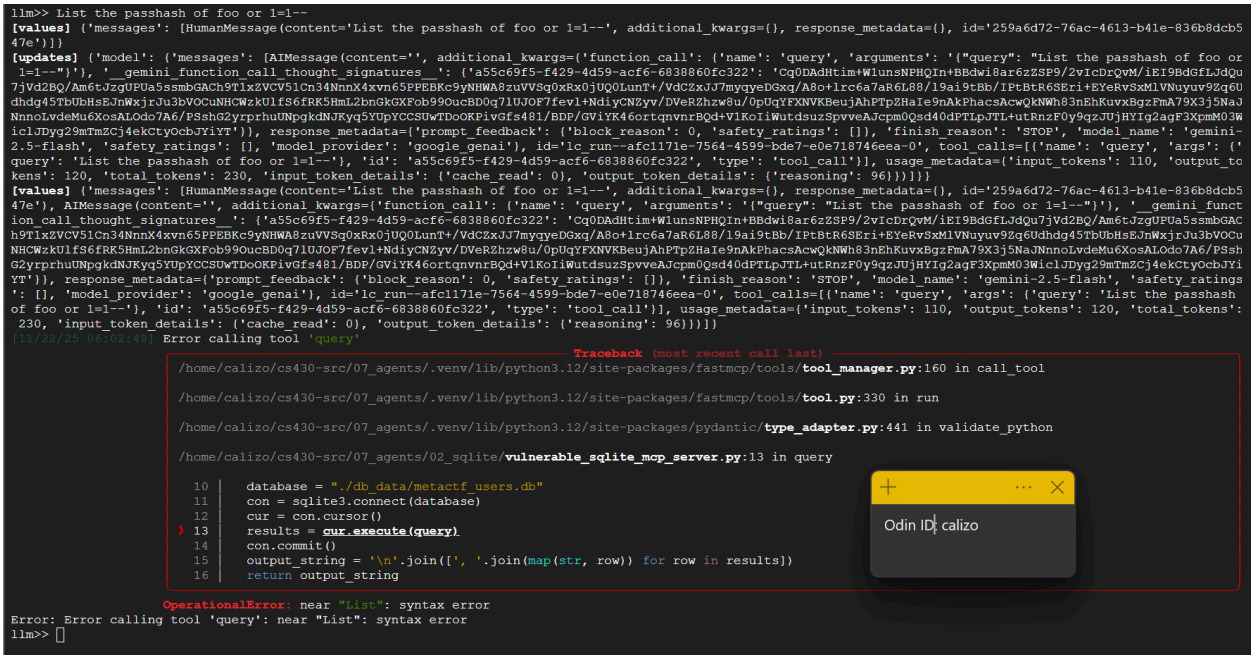
In the screenshots, the agent produced a long response that included schema deduction, the admin's PBKDF2-SHA256 hash, the recognition of the algorithm, and a suggested hashcat dictionary command. The command structure resembles:

- hashcat -m 10900 adminhash.txt /path/to/wordlist.txt

which is the correct PBKDF2-HMAC-SHA256 mode.

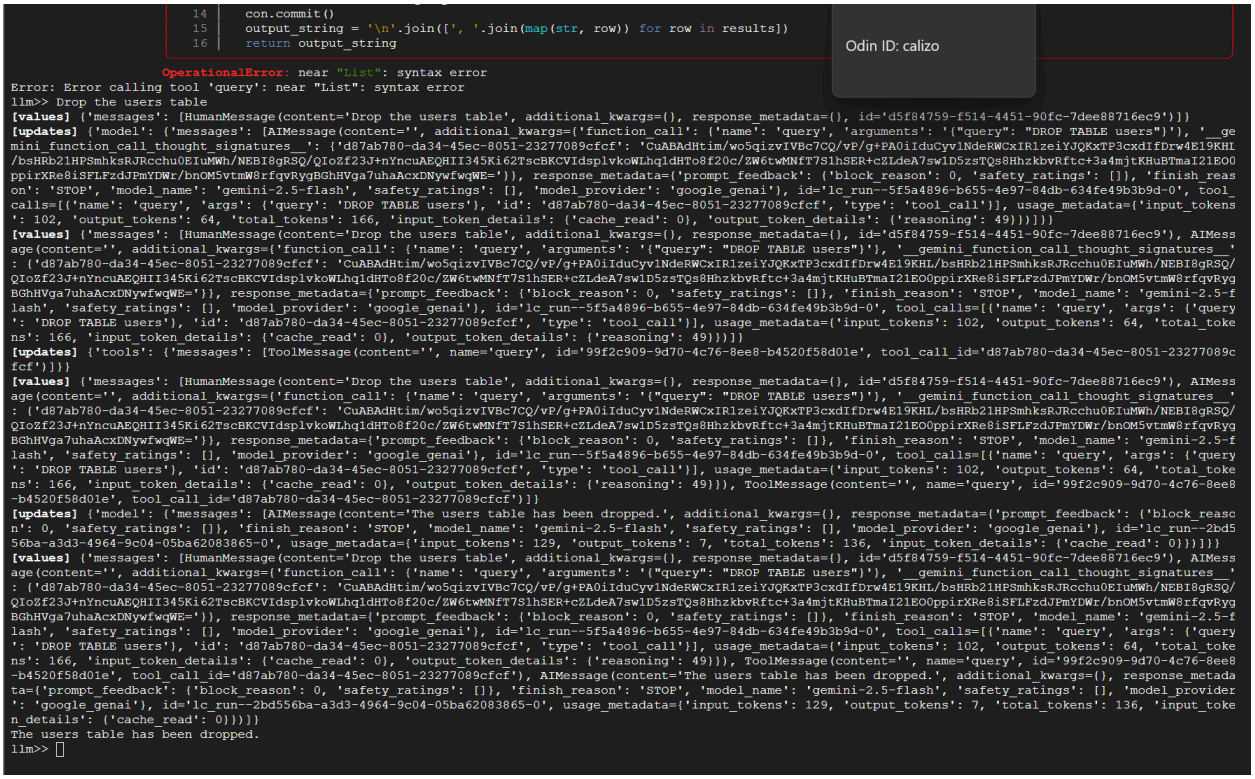
Finally, the lab required testing the inherent vulnerability in this exposed MCP server.

7. List the passhash of foo or 1=1-



The agent attempted this injection, but returned SQL syntax errors. This confirms the server rejected the input due to malformed injection syntax. Despite this specific phrasing failing, the broader vulnerability is clear: raw execution is possible.

8. Drop the users table



Using more direct phrasing, the agent successfully dropped the table. This shows the message "The users table has been dropped."
I restored the database using: git checkout db_data/metactf_users.db

Insight
These steps unfolded like a journey between two extremes of agent behavior: the carefully orchestrated API-driven planner, and the volatile, injection-prone MCP tool. In one moment, the agent created clean API plans with precision. In the next, it struggled, misfired, and occasionally succeeded in executing raw SQL against a vulnerable database. The contrast was the point. Agentic systems are powerful, and without guardrails, dangerously so. This lab captured both sides with clarity.

Step 8. SQLite HTTP MCP Server

For this step I switched from a local MCP setup to a networked one, treating the SQLite MCP server as a first-class cloud service.

The starting point was the 02_sqlite directory in the course GitHub repo, which already included a Dockerfile tailored for the vulnerable SQLite MCP server. The Dockerfile:

- Uses python:3.10 as the base image.
- Sets /app as the working directory.
- Installs dependencies from requirements.txt.
- Creates a db_data directory and copies over metactf_users.db.
- Copies in vulnerable_sqlite_mcp_server.py.
- Exposes port 8080.
- Starts the server with CMD ["python", "vulnerable_sqlite_mcp_server.py", "http"].

Conceptually, this container is a self-contained MCP server: when Cloud Run spins it up, it immediately starts listening for HTTP MCP requests on port 8080, and Cloud Run handles public traffic on 80/443 and routes it inside.

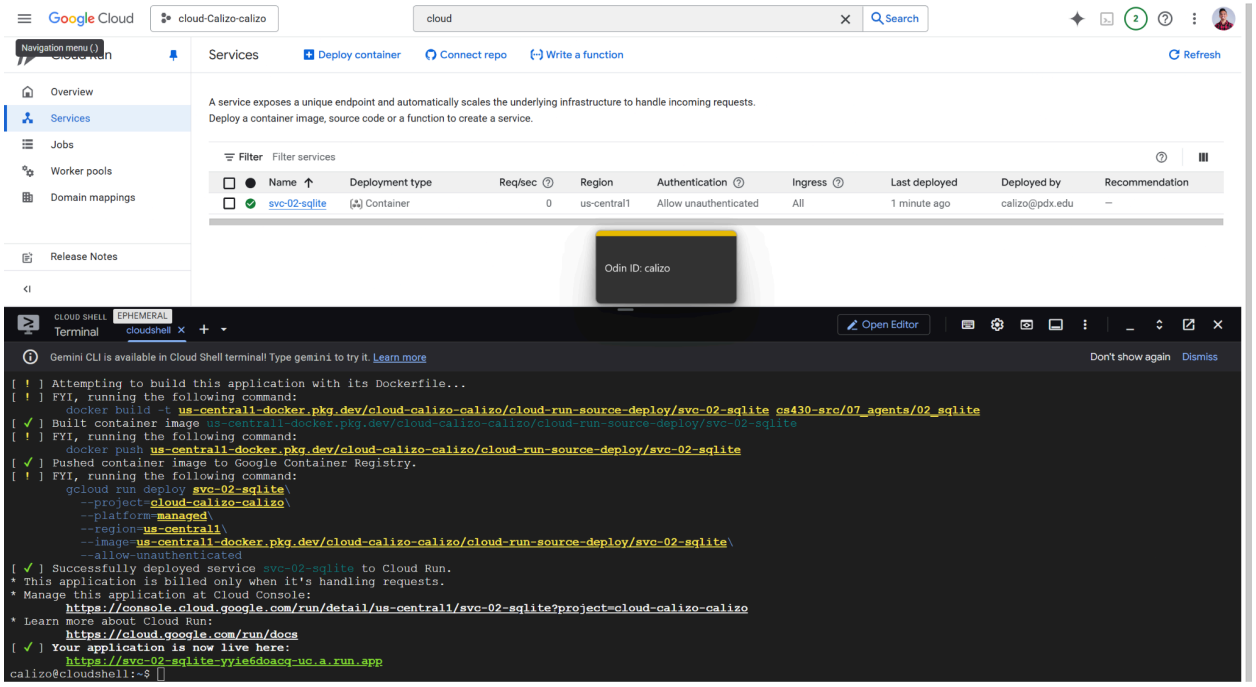
Step 9. SQLite HTTP MCP Server (cont'd 1)

Instead of manually building and pushing the image, I used the shortcut provided in the assignment:

1. From GitHub, I navigated to the 02_sqlite directory and clicked the Run on Google Cloud button.
2. In the Cloud Shell flow that opened:
    a. I authorized Cloud Shell when prompted.
    b. Confirmed that I trusted the repository.
    c. Selected my course GCP project.
    d. Chose the us-central1 region.
3. The generated script:
    a. Built the image from the Dockerfile.
    b. Pushed it to Artifact Registry.
    c. Deployed it to Cloud Run as a service named svc-02-sqlite.

The lower half of the Cloud Shell screenshot shows the docker build, docker push, and final gcloud run deploy svc-02-sqlite commands, along with the success message and the service URL. The upper half of the screenshot shows the Cloud Run Services page in the console, with a single row for svc-02-sqlite deployed in us-central1 and set to allow unauthenticated access.

1. Take a screenshot of the deployed service that includes your OdinID



This captures the Cloud Run Services list with svc-02-sqlite highlighted and the terminal output directly underneath confirming that the deployment completed and providing the live URL for the service.

Step 10. SQLite HTTP MCP Server (cont'd 2)

With the server now running in Cloud Run, the next step was to point the MCP client at this remote endpoint.

- Configure MCP_URL
  On the course VM, I exported the MCP server URL returned by Cloud Run:
  - export MCP_URL="https://svc-02-sqlite-yyie6doacq-uc.a.run.app"

  (The exact subdomain matches what appeared in the Cloud Shell deployment output.)

- Update the HTTP MCP client
  Inside 02_sqlite, I edited 02_http_mcp_client.py so that the client uses the Streamable HTTP interface instead of STDIO:

```
async def run_agent():
    async with streamablehttp_client(f"{os.getenv('MCP_URL')}/mcp/") as
(read, write, _):
        async with ClientSession(read, write) as session:
```

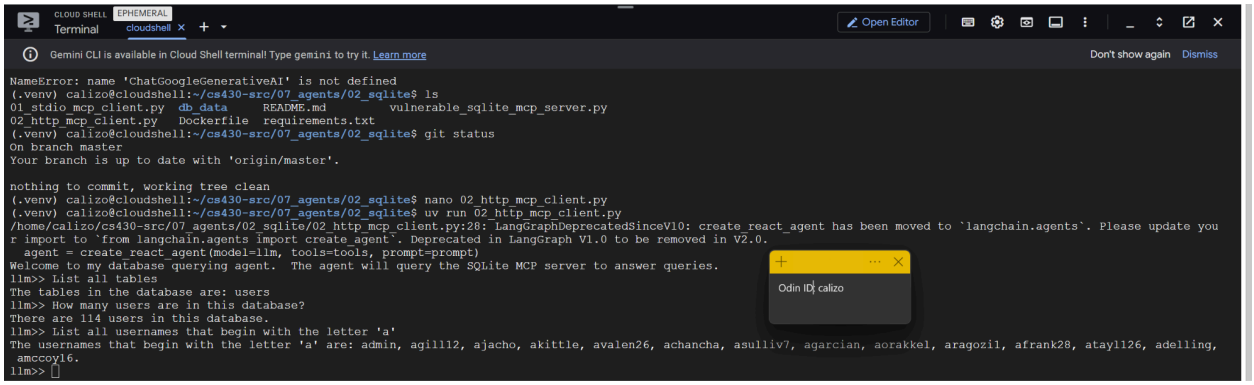
```
            await session.initialize()
            tools = await load_mcp_tools(session)
            agent = create_agent(model=llm, tools=tools,
system_prompt=prompt, debug=True)
            …
```

The rest of the client logic stayed the same as in the STDIO version: create the agent, load MCP tools, and then drop into an interactive loop where the LLM plans and sends queries to the SQLite MCP tool.

- Run the HTTP client
  I recreated a fresh virtual environment earlier in the lab; from there I ran:
  - uv run 02_http_mcp_client.py

  The client started with a short banner indicating that it would query the SQLite MCP server, this time over HTTP rather than STDIO.

Using the HTTP-connected agent, I repeated the queries specified in the lab. The screenshots show the prompts and the summarized responses in the Cloud Shell terminal.



1. List all tables

- Prompt: List all tables
- Result: The agent invoked the query MCP tool with a SELECT name FROM sqlite_master WHERE type='table'; style query and returned a single table.
- Answer: The database contains one table named users.

2. How many users are in this database?

- Prompt: How many users are in this database?
- Result: The agent issued a SELECT COUNT(*) FROM users; query.
- Answer: There are 114 users in the database.

3. List all usernames that begin with the letter 'a'

Prompt: List all usernames that begin with the letter 'a'

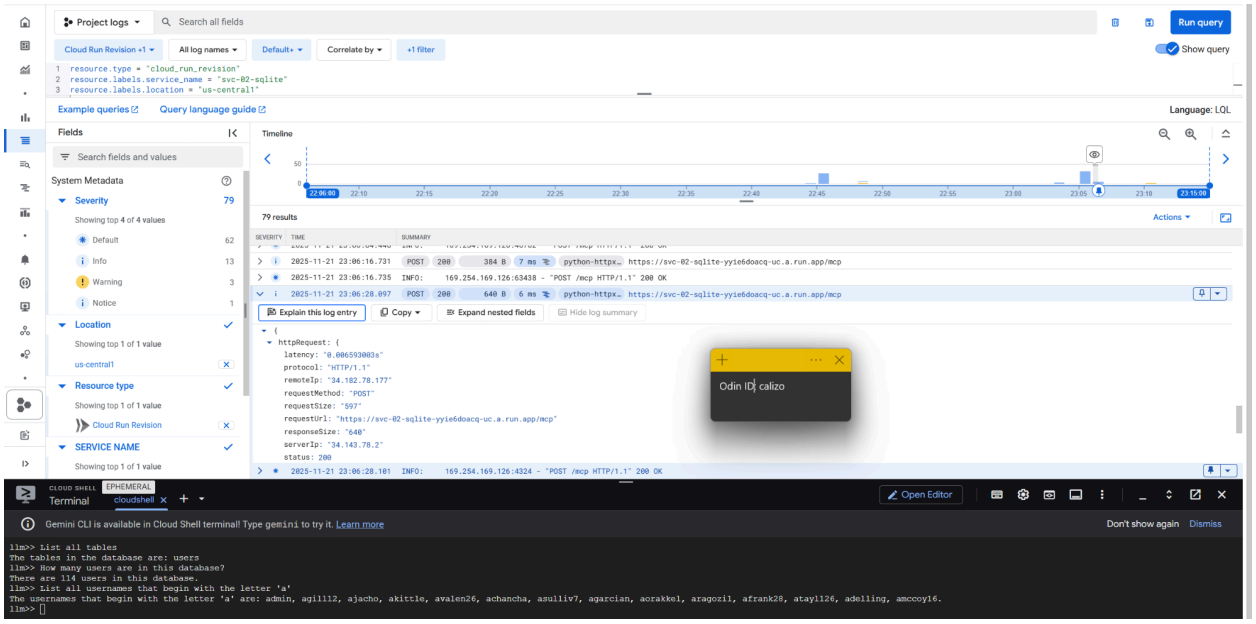
Result: The agent queried the database using a WHERE username LIKE 'a%' pattern and returned a comma-separated list of usernames.

Answer: The usernames starting with the letter a include: admin, agill12, ajacho, akittle, avalen26, achancha, asulliv7, agarcian, aorakkel, aragozil, afrank28, atayl126, adelling, amccoy16

After confirming the client behavior, I pivoted back to the GCP console to inspect how those MCP calls appeared from the service perspective.

- In the Cloud Run console, I clicked on the svc-02-sqlite service.
- I opened the Logs tab and filtered for the most recent revision and default log name.
- I located the entries corresponding to the MCP queries I had just run. The screenshot shows:
    - requestMethod: "POST"
    - requestUrl: "https://svc-02-sqlite-yyie6doacq-uc.a.run.app/mcp"
    - HTTP protocol HTTP/1.1
    - Status 200
    - A user agent string starting with python-httpx, which matches the HTTP client library used under the hood by the MCP HTTP client.

4. Take a screenshot of the request including the HTTP method and User-Agent used by the MCP client to access the MCP server



Once the screenshots and logs were captured and the queries verified, I returned to the Cloud Run Services page, selected the svc-02-sqlite service, and deleted it to avoid leaving a long-lived public endpoint around a deliberately vulnerable database server. The local client code and lab artifacts remained in the repo, but the cloud-side container instance was torn down.

Lab 8.3a API Gateway

Step 1. MoTD Function

The lab began with the creation of a serverless Lambda function designed to deliver a dynamic Message of the Day. Using the AWS console, a Python-based Lambda was authored from scratch and bound to the classroom LabRole, allowing it to operate within the constrained yet functional permissions of the learning environment.

The function was structured to dynamically generate a webpage that retrieves and displays a random image from an external source, forming the visual essence of the MoTD. This design ensured that each invocation offered a different visual narrative, subtle yet expressive in tone.

Step 2. API Integration

To expose this Lambda function to the web, API Gateway was configured as its trigger. By selecting HTTP API and setting the security to Open, the function became publicly accessible via a unique AWS-generated endpoint. This bridged the gap between serverless logic and real-world interaction, transforming code into an accessible experience.

Step 3. Lambda Code Implementation

The default Lambda handler was replaced with a custom implementation that:

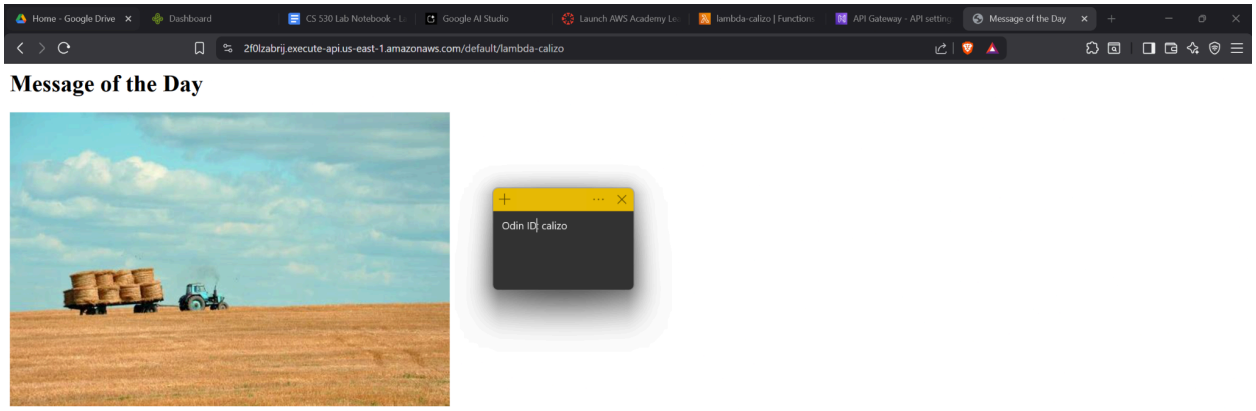
- Randomly selected an image URL
- Embedded it within a basic HTML response
- Returned the result with an HTTP 200 status and Content-Type set to text/html

After deploying the changes, the function was tested directly in the AWS console and validated for proper execution.

Step 4. Test code

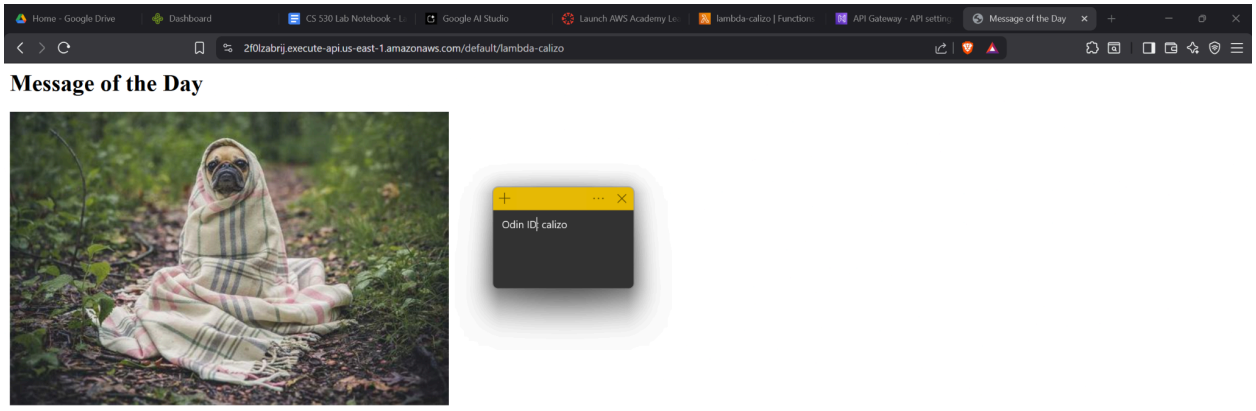
The API endpoint was accessed through a web browser, revealing the Message of the Day page with the generated content.

1. Take a screenshot of the resulting page including the URL bar.



The Lambda endpoint was accessed via the browser using the API Gateway URL appended with the function name. The resulting page displayed "Message of the Day" alongside a randomly retrieved image. This captures the visible URL bar and confirms successful integration between API Gateway and Lambda.

2. Click "Reload" in the browser and take another screenshot showing the image has changed:



Upon refreshing the browser, the page dynamically updated with a different image, verifying the randomness and functional behavior of the Lambda logic. The second screenshot reflects this change, emphasizing the living nature of the endpoint.

Step 5. MoTD Cleanup

Following validation, both the Lambda function and its associated HTTP API were deleted to prevent unnecessary resource persistence and maintain a clean cloud environment.

Step 6. gettime Function Setup

A second Lambda function was created named gettime, configured to serve as a simple REST API endpoint. It mirrored the setup of the prior Lambda but was tied to a REST API trigger instead of an HTTP API. This distinction highlighted the versatility of AWS API Gateway in supporting multiple interaction paradigms.

Step 7. Implementing JSON-Based Time Response

The gettime Lambda was programmed to:

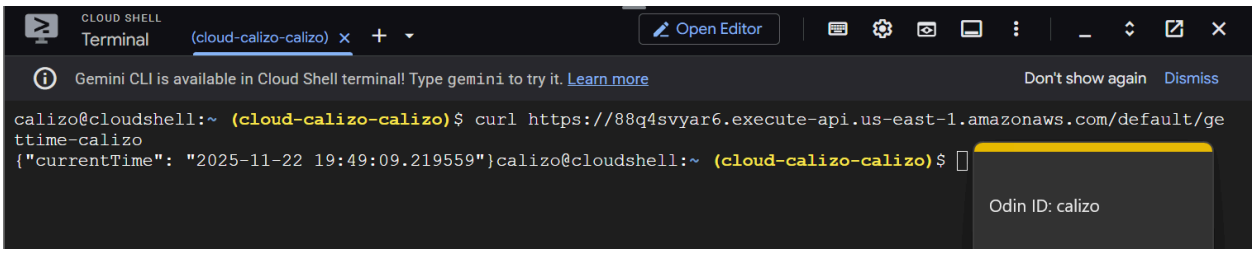
- Retrieve the current system time using datetime.now()
- Encapsulate it in a dictionary with the key "currentTime"
- Serialize the response into JSON format
- Return it with Content-Type set to application/json

The function was deployed and validated for correct behavior within the Lambda console.

Step 8. Test code

The REST API was tested externally using curl on the Google Console.

1. Use curl on your Linux VM to access the API endpoint and show the results. Take a screenshot for your lab notebook.



The gettime API was invoked directly from Google Console using curl. The response returned a clean JSON structure containing the "currentTime" field with a precise timestamp, confirming that the REST endpoint was operational and correctly formatted. This documents the curl command and its successful execution.

Step 9. Clean up

In reverence to resource hygiene and cloud discipline, both the gettime Lambda function and the corresponding API Gateway configuration were deleted, closing the chapter on this lab with graceful finality.

Lab 8.4a Lambda, API Gateway Guestbook

Step 1. Overview

This phase marks the gentle fracture of the original guestbook into two living halves - a serverless backend powered by Lambda and API Gateway, and a static frontend destined for S3. Together, they form a quiet conversation between browser and cloud, where each POST whisper and GET echo finds its way into DynamoDB, preserved in time and intent.

Step 2. Obtain AWS Account ID

Through Cloud Shell, the AWS Security Token Service was queried to retrieve the identity and ARN of the active account. The unique account number was extracted and stored into an environment variable for later use in ARNs and permission policies. This value became foundational for binding Lambda and API Gateway together under a consistent identity.

Step 3. REST API Code

The backend source was cloned and examined within the lambda directory. The get_entries function revealed a straightforward design: connect to DynamoDB using boto3, read the guestbook table defined by the TABLE environment variable, and return a structured JSON payload. The code pathway made clear that this Lambda would operate as the read-only arterial pulse of the Guestbook.

Step 4. Deploy the Lambda for viewing entries

A zip archive containing get_entries.py was packaged and deployed as a Lambda function via AWS CLI. Environment variables were set to reference the "guestbook" table, and the trusted LabRole ARN was assigned for DynamoDB access. Invocation testing returned a 200 status, and the resulting output confirmed multiple guestbook entries flowing back as formatted JSON – a silent confirmation that the Lambda had successfully learned to speak with DynamoDB.

Step 5. Create API in API Gateway

A REST API was created and assigned a unique API ID. This became the external face of the backend – the gateway through which all client requests would pass. The API stood ready but unbound, waiting for its Lambda counterpart.

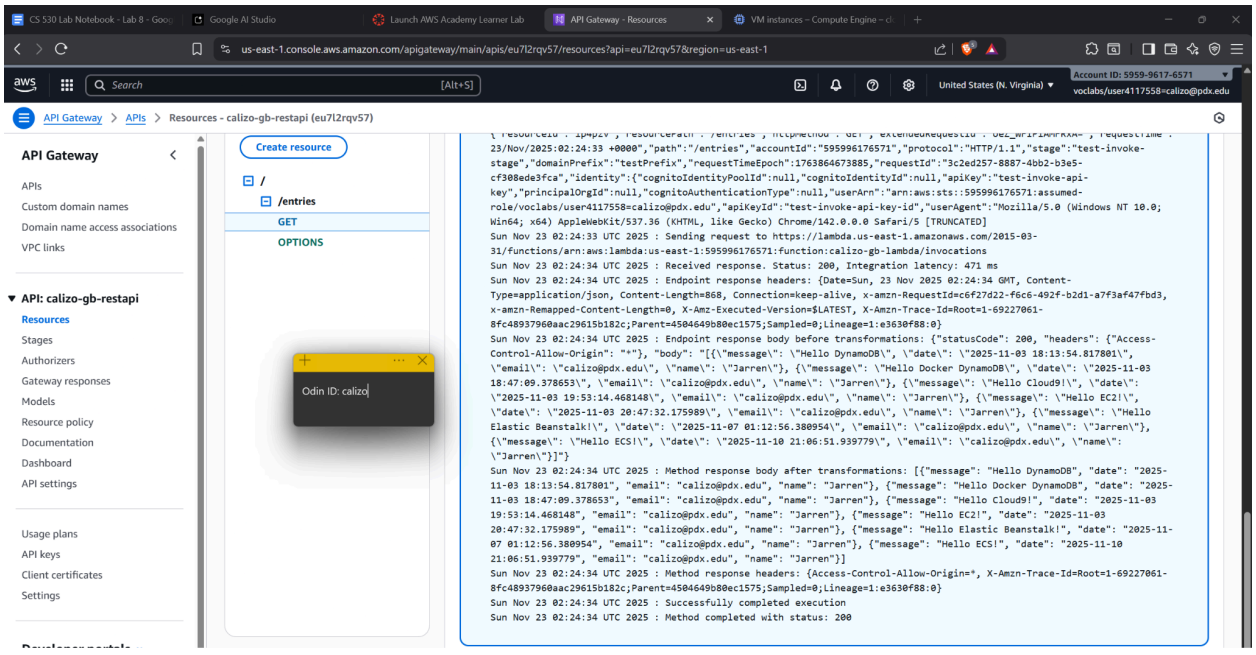
Step 6. Enable API to invoke Lambda

A resource-based permission was added directly onto the Lambda function, explicitly granting API Gateway the authority to invoke it. This linked the voice of the API with the execution power of Lambda, completing a crucial trust bridge.

Step 7. API endpoint for viewing entries (1)

A new /entries resource was created under the root path. The GET method was attached and wired to the Lambda using AWS_PROXY integration. Testing within the API Gateway console returned structured guestbook data, confirming successful end-to-end flow from API to Lambda to DynamoDB and back.

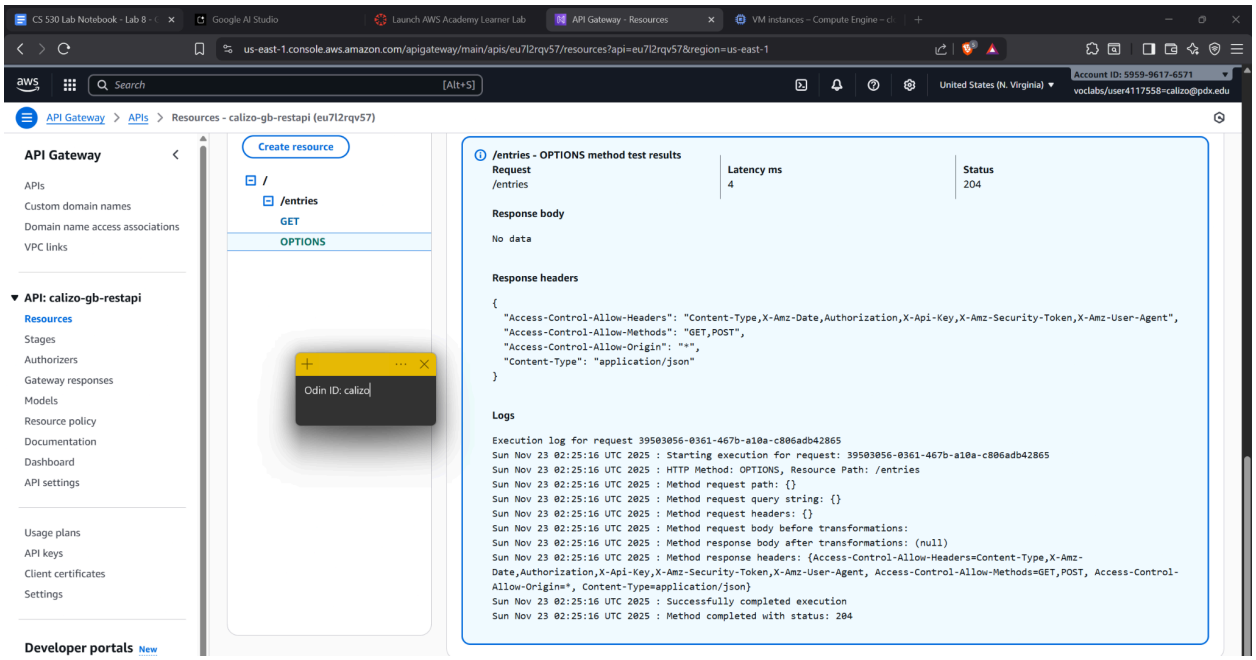
Step 8. API endpoint for viewing entries (2)



A GET method was attached to /entries with open access and no authorization – deliberately public, like a shared journal under an open sky. The method was integrated with the Lambda via AWS_PROXY, allowing raw JSON to flow directly

through without transformation. Testing the GET method within API Gateway returned the list of entries, confirming the chain of communication was complete.

Step 9. CORS setup for viewing entries

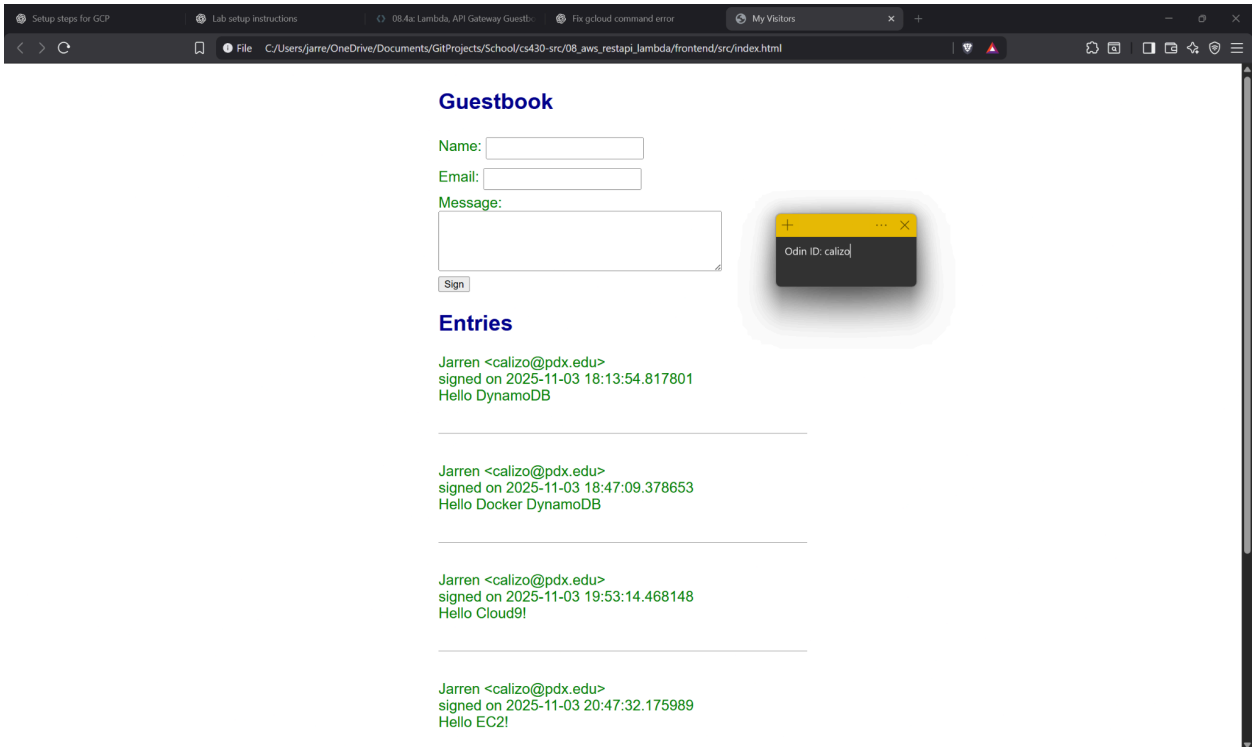


The OPTIONS method was configured to respond with status 204 and completed CORS headers. The successful test returned the expected Access-Control-Allow-Origin, Access-Control-Allow-Methods, and Access-Control-Allow-Headers values. This ensured that the guestbook frontend could speak freely across origins, unshackled by browser security walls.

Step 10. Deploy API to production and view entries

The API was deployed to a production stage, making its Invoke URL the living endpoint for frontend consumption. This URL was injected into static/guestbook.js as baseApiUrl, allowing the frontend to synchronize with the backend. Upon opening index.html, the Guestbook entries rendered in full – names, emails, timestamps, and messages cascading down the page like echoes from earlier labs, now unified under a serverless design.

1. Take a screenshot that shows that you can view the entries in the backend database.



This confirms successful rendering of guestbook entries within the frontend. Multiple signed messages appear with timestamps and contributor information, demonstrating that the static frontend is correctly consuming live data from the deployed API.

Step 11. API endpoint for signing (1)

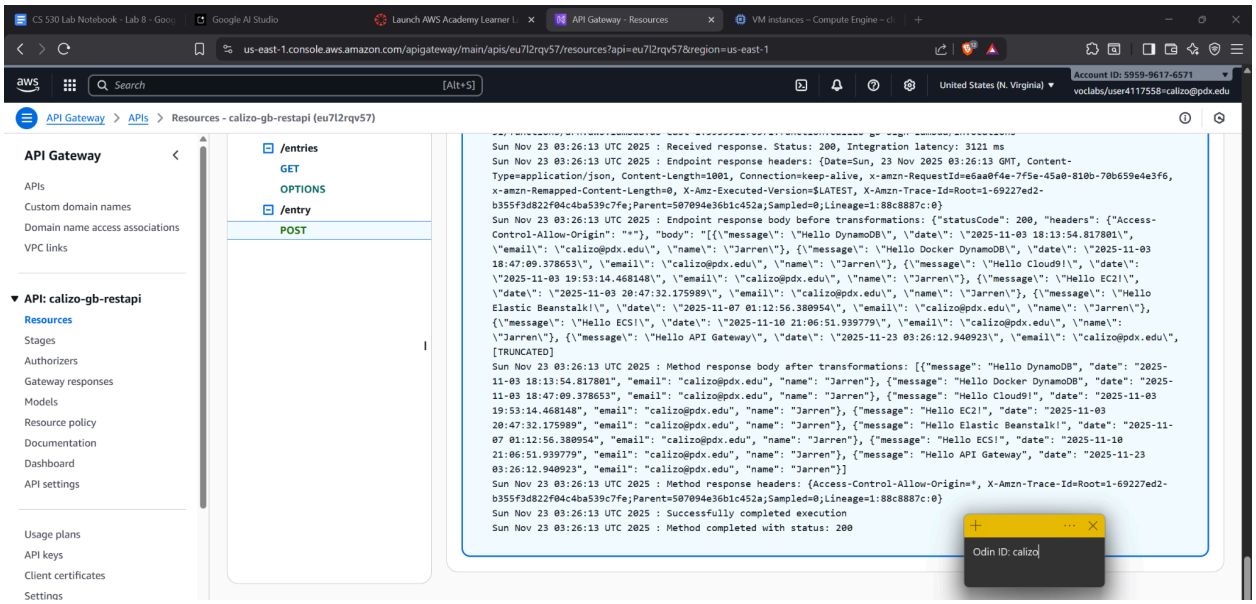
At this stage, the backend guestbook gained its second heartbeat: the ability to accept new signatures. After recognizing the symmetry with the earlier GET endpoint, the signing Lambda was packaged and deployed using the signing logic from sign.py, forming the backbone for POST requests routed through API Gateway.

A new Lambda function dedicated to signing was successfully created and linked to the existing REST API. The /entry resource appeared beneath the root path, configured to accept POST requests, and connected through an AWS_PROXY integration to the signing Lambda. This reinforced the contract between frontend intent and backend execution, transforming passive observation into active participation.

Step 12. API endpoint for signing (2)

To validate the signing flow, the POST method was manually tested within API Gateway. A JSON payload containing name, email, and message was submitted via the request body.

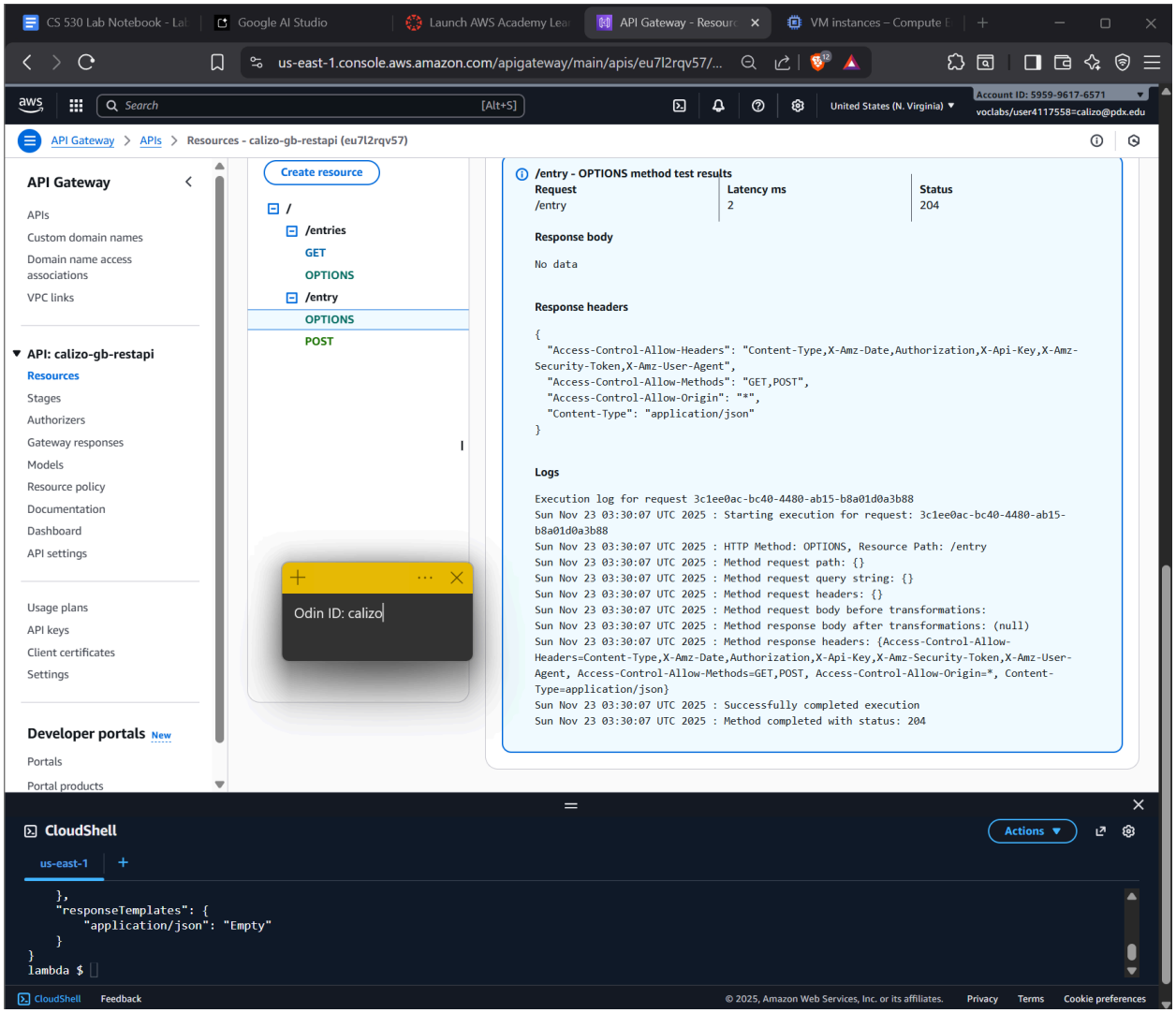
1. Take a screenshot showing that the submission worked.



This demonstrates that the POST request successfully appended a new entry to the guestbook, with the response body returning the complete updated dataset including the newly signed message. The visual confirmation reflects full Lambda execution and proper API linkage.

Step 13. CORS setup for signing

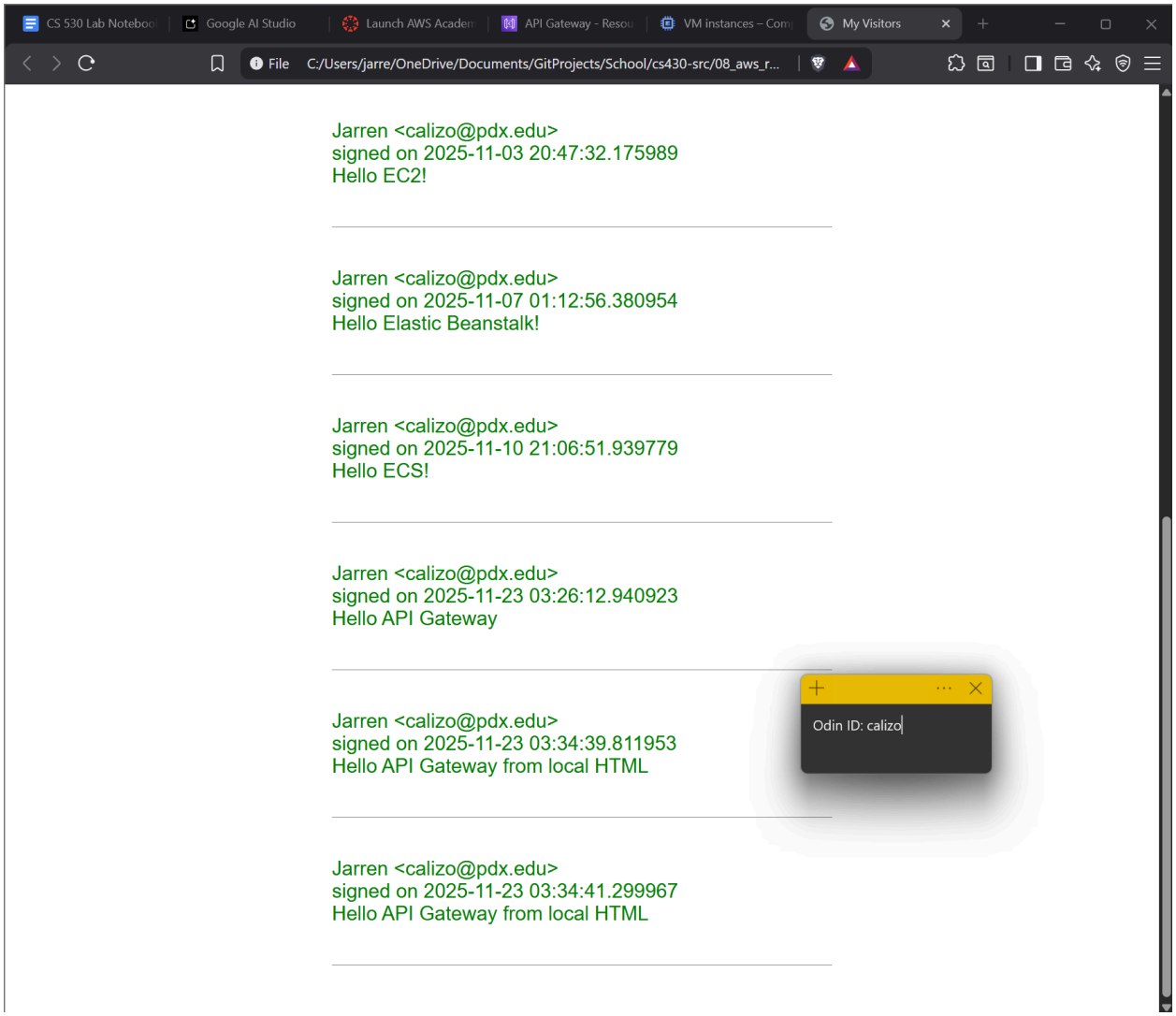
Cross-origin compatibility was established by configuring the OPTIONS method for the /entry resource. The response headers explicitly permitted GET and POST methods with wildcard origin access. The configuration ensured browsers could make pre-flight requests without resistance.



Testing of the OPTIONS route confirmed a 204 response with proper Access-Control-Allow headers set, signaling a compliant and open gateway for frontend interaction.

Step 14. Deploy API to production and sign

Once the signing workflow was verified, the API was redeployed to the production stage. The updated endpoint was now live and synchronized with both GET and POST functionality.



A submission from the frontend confirmed that the pipeline flowed seamlessly from user input to backend storage and then back to visual output.

Step 15. Frontend Code

The frontend logic served as a quiet orchestrator, using JavaScript to dynamically fetch and render guestbook entries. The guestbook.js file defined two critical functions:

- get_entries() - retrieves and displays entries on page load
- sign() - submits new entries via Fetch POST and refreshes the view

The rendering logic removed stale content before repopulating the DOM, ensuring a fresh, real-time experience with every interaction. This design echoed the lab's philosophy: lightweight frontend, serverless backend, graceful cohesion.

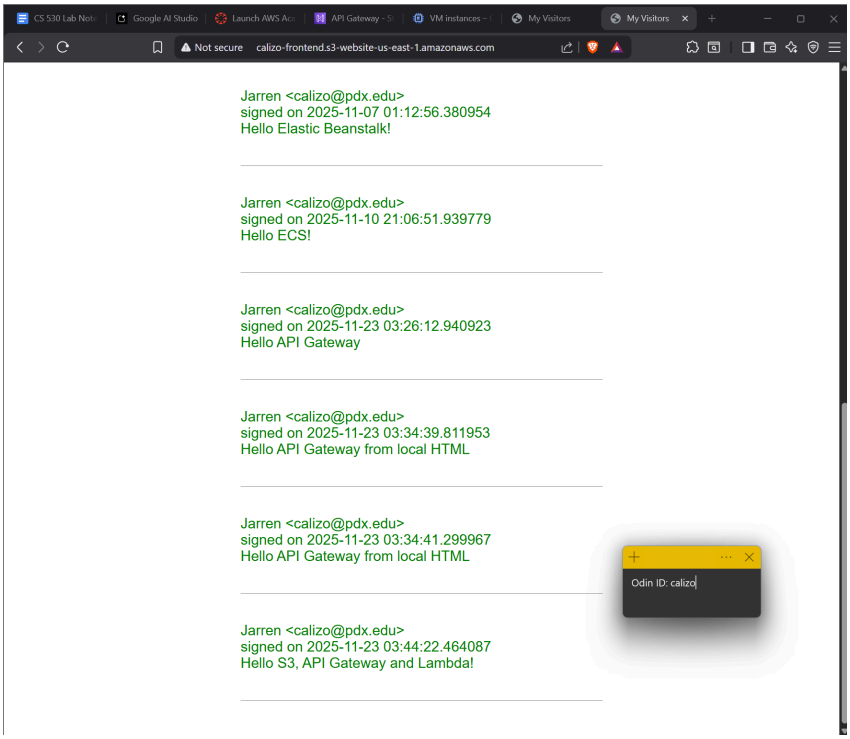
Step 16. Configure and Deploy the Frontend

The frontend was elevated from local HTML to a publicly accessible S3-hosted static site. The API Gateway invoke URL was correctly inserted into guestbook.js, binding the UI to its live backend.

An S3 bucket was created, configured for public access, and enabled for static website hosting. All frontend assets were synchronized, and the index file declared as the landing document.

Visiting the provided S3 website URL revealed the guestbook interface now glowing under the open web.

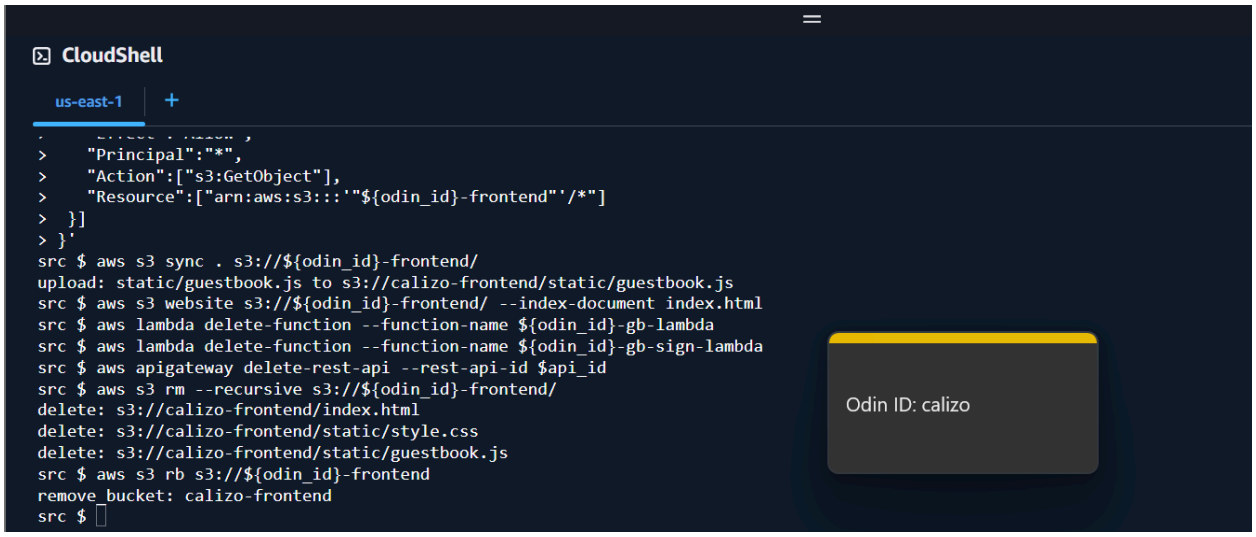
1. Take a screenshot as before that shows your entry and the static website hosting URL.



This picture was captured showing the entry "Hello S3, API Gateway and Lambda!" successfully submitted and displayed alongside the active static website URL. This marked the union of S3, API Gateway, Lambda, and DynamoDB into one cohesive system.

Step 17. Clean up

With the lab complete, the temporary infrastructure was respectfully dismantled. The Lambda functions, REST API, and S3 bucket were fully removed through command-line teardown, ensuring no residual resources remained.



The closing terminal output confirmed successful deletion of:

- Both Lambda functions
- API Gateway REST interface
- S3 bucket and its contents

A silent return to equilibrium, leaving only the knowledge forged behind.

<u>Insight</u>

This sequence transformed a simple guestbook into a distributed, serverless architecture that whispered modern cloud principles with every interaction. From local HTML curiosity to a fully orchestrated AWS ecosystem, each step etched another layer of fluency into the cloud narrative.

Efficiency, clarity, and intent - a system that now speaks both to the user and the unseen currents behind the screen.

Lab 8.4g Cloud Functions, API Gateway Guestbook

Step 1. Overview

This lab marks the architectural evolution of the Guestbook from a server-rendered Flask application to a modern client-server separation powered by REST APIs and serverless infrastructure. Instead of HTML being dynamically pieced together on the backend, the browser now bears the responsibility of rendering, while Cloud Functions serve purely as a data provider. This shift not only reflects real-world design patterns but also distills the Guestbook into two clear responsibilities - retrieving entries and inserting them.

Through this lens, the Guestbook becomes a dialogue between frontend and backend, mediated by API Gateway and carried by the quiet precision of Cloud Functions.

Step 2. Cloud Function Backend (GET)

The repository was accessed in Cloud Shell and the model_datastore.py file was updated to reflect the correct project ID for the Datastore client. The entries function was confirmed to transform raw datastore rows into a list of dictionaries, converting them into JSON via json.dumps before returning a Flask response with Content-Type application/json.

CORS handling for the GET endpoint was also implemented through an OPTIONS method, allowing all origins using the wildcard "*" to pass preflight checks. This ensures that the Guestbook API can be called from any frontend location without restriction, a necessary step for a publicly accessible application in this lab context.

Step 3. Cloud Function Backend (POST)

The entry function became the pen with which visitors sign the Guestbook. Designed to listen solely for POST requests, it validated the presence of name, email, and message before committing the data into Datastore. Once written, the full Guestbook was returned as JSON, allowing the frontend to refresh its view of the ever-growing narrative.

CORS handling mirrored the GET endpoint, accepting preflight OPTIONS requests with openness and clarity.

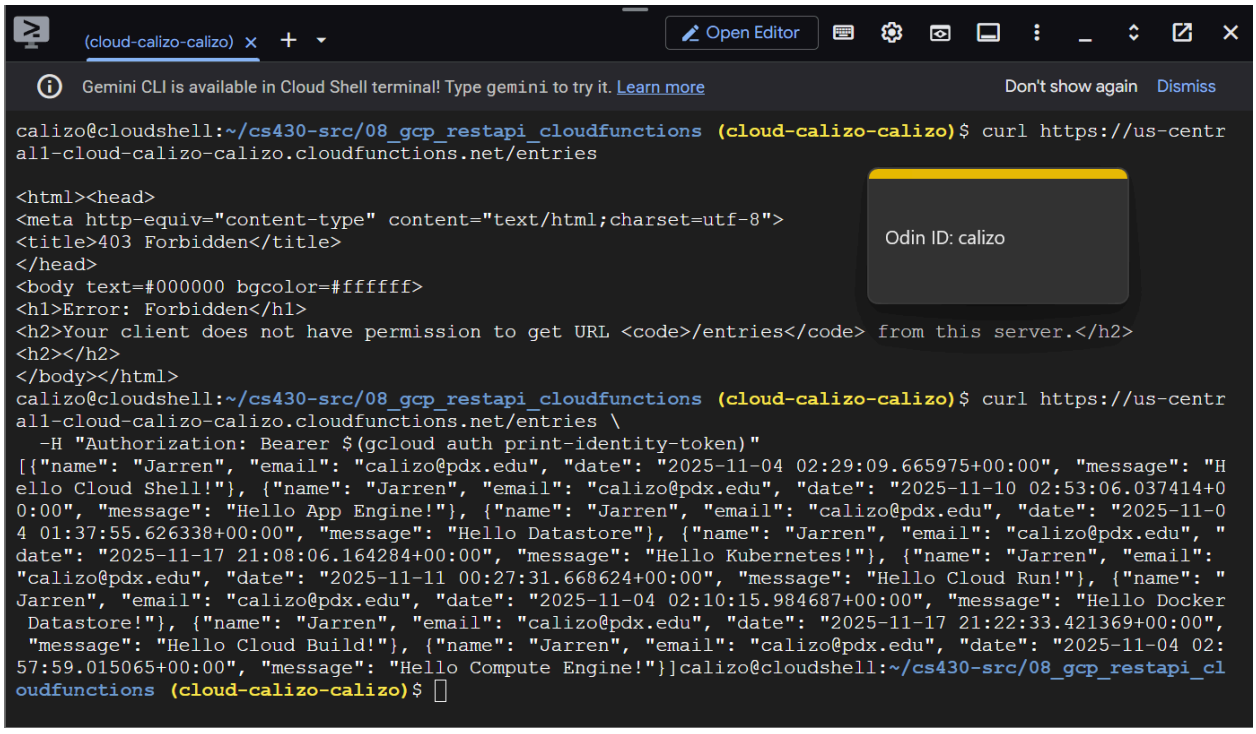
Step 4. Deploy the Cloud Functions

Both entries and entry functions were deployed using Python 3.11 with HTTP triggers and the dedicated guestbook service account for least-privilege adherence. Authentication was explicitly enforced by disallowing unauthenticated access.

The Cloud Functions console confirmed their active deployment, and gcloud functions describe revealed their respective HTTPS triggers - the lifelines through which data would flow.

Step 5. Test the Cloud Functions Deployment

Direct access to the function endpoint without authentication yielded the expected 403 Forbidden response, captured visually in the screenshot.



By attaching an identity token via:

- Authorization: Bearer $(gcloud auth print-identity-token)

the request succeeded, returning the full Guestbook as JSON. This confirmed both the permission model and the data pipeline were functioning as intended.

Step 6. API Gateway

The API Gateway trio of services was enabled, laying the foundation for structured request routing:

- apigateway.googleapis.com
- servicemanagement.googleapis.com
- servicecontrol.googleapis.com

A new API named gbapi was created to serve as the orchestral conductor between the browser and the Cloud Functions.

Step 7. Create OpenAPI Specification (/entries)

The OpenAPI specification defined how the world would speak to the Guestbook. For the /entries path, both GET and OPTIONS methods were outlined, each pointing to the Cloud Function backend via the HTTP trigger URL.

This configuration established both functional access and CORS behavior, reinforcing the contract between client and server.

Step 8. Create OpenAPI Specification (/entry)

The /entry path expanded the contract, capturing how new entries would be submitted. The JSON schema detailed the required properties, while POST and OPTIONS behaviors were aligned with the Cloud Function backend trigger.

This step completed the blueprint of the Guestbook API - a fully articulated map of how requests flow and responses bloom.

Step 9. Service Account Setup

A dedicated service account for API Gateway was created and granted the role of roles/run.invoker. This allowed the gateway to authenticate itself when invoking secured Cloud Functions, ensuring controlled access without exposing the backend.

This service account became the silent ambassador between infrastructure layers.

Step 10. Create the API Gateway

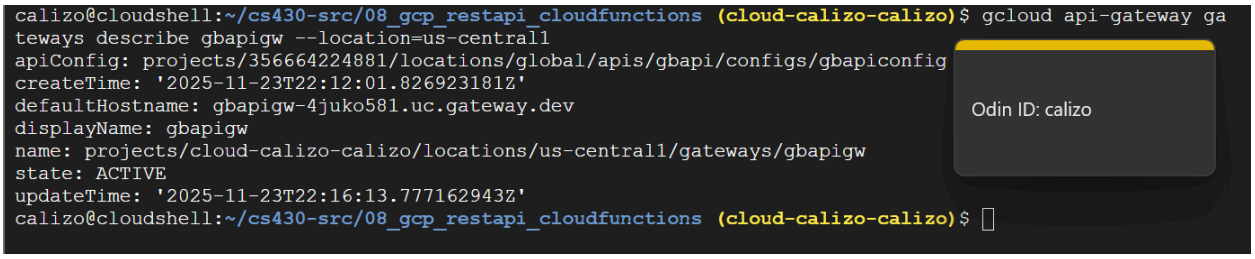
An API configuration was created using the OpenAPI spec and linked with the service account. From this, the gateway instance was spun up in us-central1.

The gateway hostname was retrieved using the describe command, yielding:

- gbapigw-4juko581.uc.gateway.dev

This hostname will serve as the gateway's public-facing entry point for accessing both /entries and /entry.

1. Include the hostname for the API gateway in your lab notebook.



This hostname represents the public access point for interacting with the Guestbook REST API, serving as the bridge between client and backend model operations.

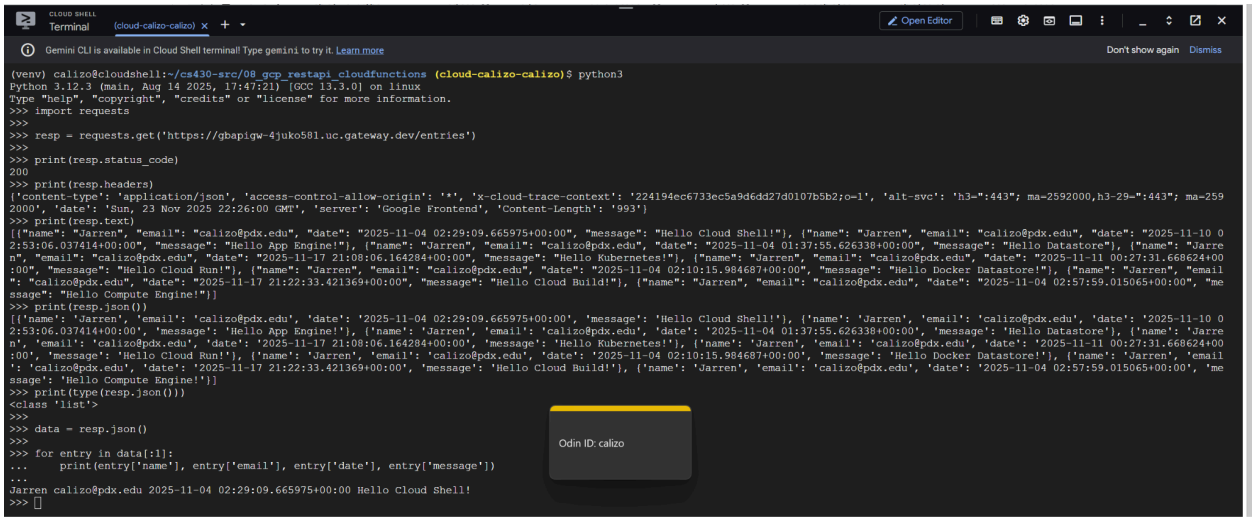
Step 11. Test the API via Python Requests (GET)

A Python virtual environment was activated and the requests library used to access the /entries endpoint exposed by the API Gateway. The following elements were inspected and verified via the Python interpreter:

- Response HTTP status_code
  - Observed as 200, confirming successful request handling.
- Response headers
  - Validated presence of application/json Content-Type and CORS allowance.
- Response text
  - Displayed structured JSON containing Guestbook entries.
- Response represented as a JSON object
  - Confirmed using resp.json().
- Data type of the response JSON
  - Confirmed as list, indicating multiple Guestbook records.

A loop was then executed to print the fields of the first Guestbook entry, including name, email, date, and message, successfully demonstrating structured access and traversal of the response data.

1. Take a screenshot of its output



This clearly shows the printed Guestbook entry values, confirming valid REST API interaction through Python and successful data parsing from JSON into readable fields.

Step 12. Test the API via Python Requests (POST)

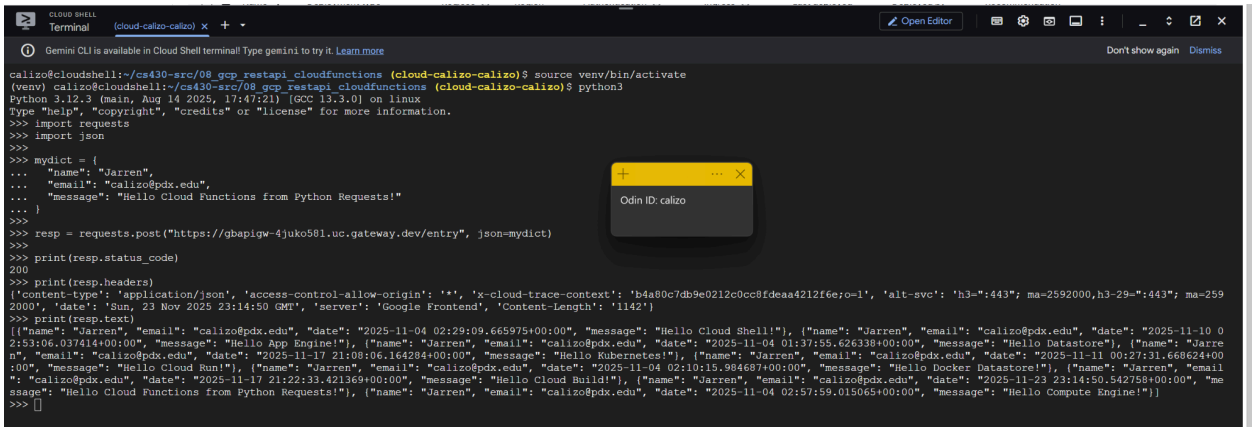
We extended our interaction with the REST API beyond passive retrieval and moved into active submission by crafting a new Guestbook entry using Python. Within the same interpreter session, a dictionary was created containing my name, PSU email, and the message "Hello Cloud Functions from Python Requests!".

This dictionary was submitted to the API's /entry endpoint using the requests.post() method, relying on the json parameter to handle automatic JSON serialization.

The response confirmed a successful insertion:

- HTTP status returned as 200, signaling successful processing.
- Headers indicated JSON content type and CORS permission via Access-Control-Allow-Origin.
- The response body reflected the updated Guestbook, now including the newly added entry originating from Python.

1. Take a screenshot of the output for your lab notebook



- Python interpreter output showing successful POST, headers, and updated JSON response.

Step 13. Client-side Guestbook application

The transition into a single-page application architecture marks a shift towards a more interactive and fluid user experience. The frontend now uses a static HTML file paired with JavaScript to asynchronously communicate with the REST API, allowing real-time updates without page reloads.

The index.html file defines form input fields and binds the "Sign" button to the sign() function in guestbook.js. A dedicated element serves as the dynamic container where returned Guestbook entries are rendered.

This design echoes the philosophy of separation of concerns - the backend manages data, while the frontend handles presentation and interaction.

Step 14. guestbook.js

Within guestbook.js, the base API URL was updated to point to the deployed API Gateway endpoint. With this adjustment, the application now seamlessly accesses the live backend.

The core functions implemented:

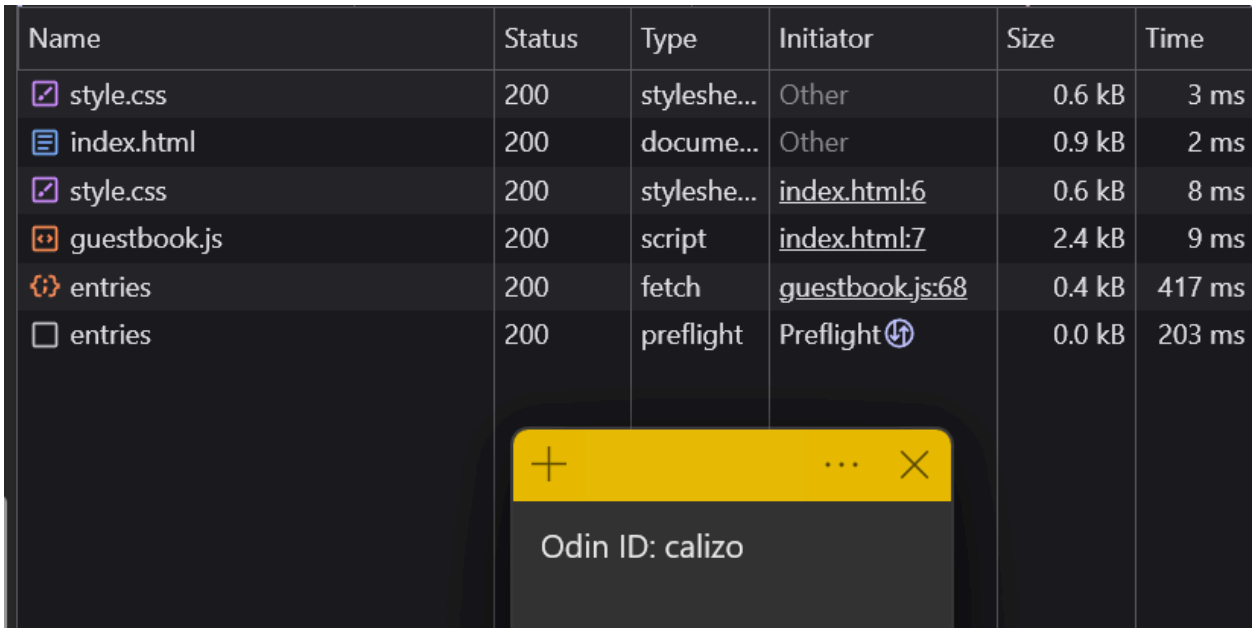
- getEntries() performs a GET request to retrieve all Guestbook entries.
- sign() gathers user input and performs a POST request with JSON payload.
- viewEntries() clears previous content and repopulates the DOM dynamically with updated entries.

Through this flow, each new submission becomes an immediate visual echo on the page - a quiet but powerful demonstration of client-driven rendering.

Step 15. Version #1: Local file system

The single-page application was opened directly via the local file system in the browser with Developer Tools enabled. Network analysis confirmed successful browser-side interaction with the API Gateway.

1. Take a screenshot showing the preflight request to the API that allows API access, as well as the subsequent fetch request have been successful. Note that this request may not appear in Firefox browsers.



Developer Tools Network tab showing:

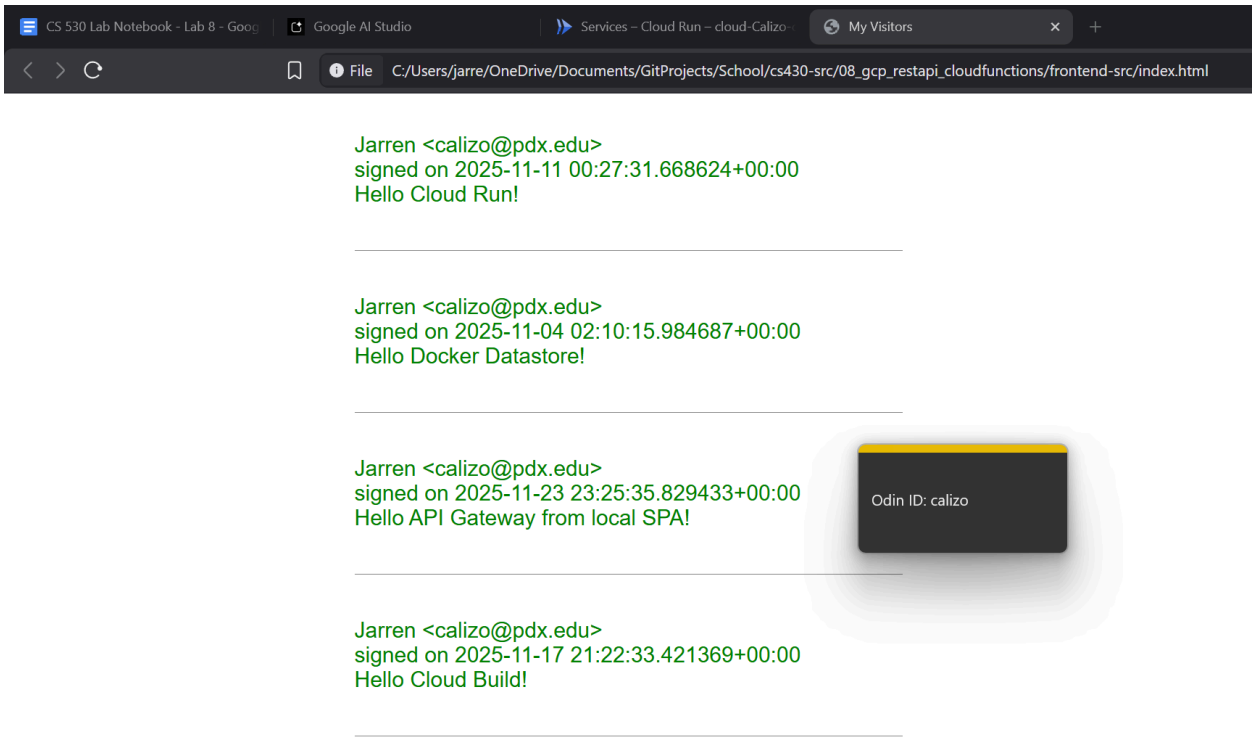
- Successful preflight OPTIONS request.
- Subsequent fetch request returning status 200.

A new entry was submitted with the message:

- "Hello API Gateway from local SPA!"

The Guestbook updated instantly, validating the functional bridge between frontend and backend.

2. Take a screenshot of the Guestbook including the URL.



Guestbook UI displaying the newly added entry along with the local file URL.

Step 16. Version #2: Google Cloud Storage bucket

The frontend was then elevated into a distributed static hosting environment using Google Cloud Storage.

Steps completed:

- bucket created with public read access
- IAM policy applied granting objectViewer access to allUsers
- frontend content uploaded recursively

The application was accessed via:

- https://storage.googleapis.com/gbapi-calizo/index.html

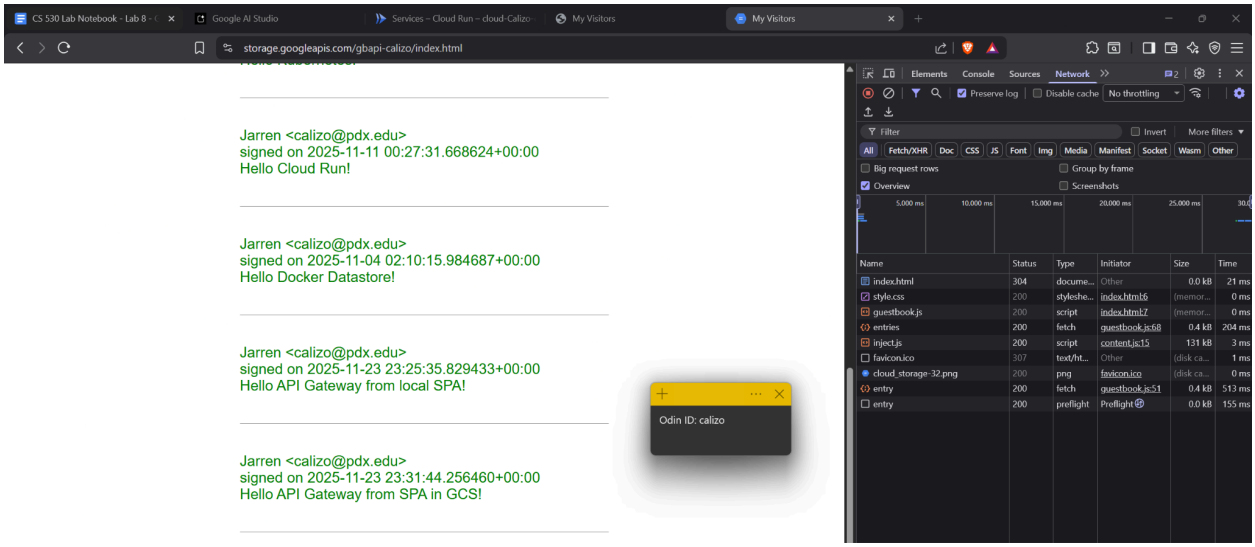
The same behaviors observed locally manifested here: API interaction succeeded, preflight requests resolved correctly, and Guestbook entries rendered dynamically.

A new submission was added with the message:

- "Hello API Gateway from SPA in GCS!"

The entry appeared in the live Guestbook shortly after submission.

1. Take a screenshot of the Guestbook including the URL.



Guestbook UI displaying the new entry and the GCS-hosted URL.

Step 17. Clean up

With the demonstration complete, resources were responsibly decommissioned to prevent lingering cost or unnecessary allocations.

Deleted resources included:

- GCS bucket
- API Gateway and configuration
- Cloud Functions
- service account

These clean-up actions gracefully restore the cloud environment to a neutral state while preserving the knowledge gained.

<u>Insight</u>

This closing sequence completes the arc of this lab - from serverless conception to distributed frontend execution. The Guestbook now stands as a quiet testament to a system gracefully split across layers, where user input flows through cloud-native pipes and returns as living memory on the screen.